

SUMMER TRAINING/INTERNSHIP REPORT

(June – July 2026)

Submitted by:

Name & Registration No. : Gaurav Kumar—12408714

Name & Registration No. : Ekagrah sharma—12409619

Name & Registration No. : Mohit—12410323

Name & Registration No. : Tanuj ahuja—12409092

Name & Registration No. : sudalai krishna—12419774

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Contents of This Listing

1. Student.h (C++ Header)
2. Student.cpp (C++ Source)
3. AttendanceSystem.h (C++ Header)
4. AttendanceSystem.cpp (C++ Source)
5. FileManager.h (C++ Header)
6. Search.h (C++ Header)
7. MergeSort.h (C++ Header)
8. main.cpp (C++ Source)
9. index.html (HTML)
10. style.css (CSS)
11. app.js (JavaScript)
12. students.txt (Data File)

Student.h

C++ Header — 38 lines

```
1 | #ifndef STUDENT_H
2 | #define STUDENT_H
3 |
4 | #include <string>
5 |
6 | class Student {
7 | private:
8 |     int rollNo;
9 |     std::string name;
10 |     std::string department;
11 |     int totalClasses;
12 |     int attendedClasses;
13 |
14 | public:
15 |     // Constructors
16 |     Student();
17 |     Student(int rollNo, std::string name, std::string department, int totalClasses, int attendedClasses);
18 |
19 |     // Getters
20 |     int getRollNo() const;
21 |     std::string getName() const;
22 |     std::string getDepartment() const;
23 |     int getTotalClasses() const;
24 |     int getAttendedClasses() const;
25 |
26 |     // Setters
27 |     void setRollNo(int rollNo);
28 |     void setName(std::string name);
29 |     void setDepartment(std::string department);
30 |     void setTotalClasses(int totalClasses);
31 |     void setAttendedClasses(int attendedClasses);
32 |
33 |     // Utility Methods
34 |     double getAttendancePercentage() const;
35 |     void printReport() const;
36 | };
37 |
38 | #endif // STUDENT_H
```

Student.cpp

C++ Source — 50 lines

```

1 | #include "Student.h"
2 | #include <iostream>
3 | #include <iomanip>
4 |
5 | // Default Constructor
6 | // Time Complexity: O(1)
7 | Student::Student() : rollNo(0), name(""), department(""), totalClasses(0), attendedClasses(0) {}
8 |
9 | // Parameterized Constructor
10 | // Time Complexity: O(1)
11 | Student::Student(int roll, std::string n, std::string dept, int total, int attended)
12 |     : rollNo(roll), name(n), department(dept), totalClasses(total), attendedClasses(attended) {}
13 |
14 | // Getters (Time Complexity: O(1))
15 | int Student::getRollNo() const { return rollNo; }
16 | std::string Student::getName() const { return name; }
17 | std::string Student::getDepartment() const { return department; }
18 | int Student::getTotalClasses() const { return totalClasses; }
19 | int Student::getAttendedClasses() const { return attendedClasses; }
20 |
21 | // Setters (Time Complexity: O(1))
22 | void Student::setRollNo(int roll) { rollNo = roll; }
23 | void Student::setName(std::string n) { name = n; }
24 | void Student::setDepartment(std::string dept) { department = dept; }
25 | void Student::setTotalClasses(int total) { totalClasses = total; }
26 | void Student::setAttendedClasses(int attended) { attendedClasses = attended; }
27 |
28 | // Safely calculate attendance percentage
29 | // Time Complexity: O(1)
30 | double Student::getAttendancePercentage() const {
31 |     if (totalClasses <= 0) {
32 |         return 0.0;
33 |     }
34 |     return (static_cast<double>(attendedClasses) / totalClasses) * 100.0;
35 | }
36 |
37 | // Print formatted student report card row
38 | // Time Complexity: O(1)
39 | void Student::printReport() const {
40 |     double pct = getAttendancePercentage();
41 |     std::string status = (pct >= 75.0) ? "Eligible" : "Not Eligible";
42 |
43 |     std::cout << std::left << std::setw(10) << rollNo
44 |         << std::setw(20) << name
45 |         << std::setw(15) << department
46 |         << std::setw(15) << totalClasses
47 |         << std::setw(15) << attendedClasses
48 |         << std::fixed << std::setprecision(2) << std::setw(15) << pct
49 |         << std::setw(15) << status << "\n";
50 | }

```

AttendanceSystem.h

C++ Header — 47 lines

```

1 | #ifndef ATTENDANCE_SYSTEM_H
2 | #define ATTENDANCE_SYSTEM_H
3 |
4 | #include <vector>
5 | #include <unordered_map>
6 | #include <string>
7 | #include <queue>
8 | #include <stack>
9 | #include <utility>
10 | #include "Student.h"
11 |
12 | class AttendanceSystem {
13 | private:
14 |     std::vector<Student> students;
15 |     std::unordered_map<int, int> rollIndex; // rollNo -> index in students vector
16 |     std::stack<std::pair<int, std::string>> undoStack; // records attendance actions for undo (rollNo,
"Present"/"Absent")
17 |
18 |     // Helper to rebuild rollIndex map after vector modifications
19 |     void rebuildRollIndex();
20 |
21 | public:
22 |     AttendanceSystem();
23 |
24 |     // Load from and save to disk wrappers
25 |     void loadFromFile();
26 |     void saveToFile() const;
27 |
28 |     // Getters for student list
29 |     const std::vector<Student>& getStudents() const;
30 |     std::vector<Student>& getStudentsMutable();
31 |
32 |     // Operations
33 |     bool addStudent(const Student& s);
34 |     bool deleteStudent(int rollNo);
35 |     bool updateStudent(int rollNo, const std::string& newName, const std::string& newDept, int newTotal, int
newAttended);
36 |     void markAttendanceSession(const std::vector<int>& rollNumbers);
37 |     bool undoLastAttendance();
38 |
39 |     // Reports
40 |     void displayAllStudents() const;
41 |     void lowAttendanceReport() const;
42 |
43 |     // Helper to find a student's index by roll number
44 |     int findIndexByRollNo(int rollNo) const;
45 | };
46 |
47 | #endif // ATTENDANCE_SYSTEM_H

```

AttendanceSystem.cpp

C++ Source — 285 lines

```

1 | #include "AttendanceSystem.h"
2 | #include "FileManager.h"
3 | #include <iostream>
4 | #include <iomanip>
5 | #include <cmath>
6 |
7 |
8 | // Constructor
9 | AttendanceSystem::AttendanceSystem() {
10 |     loadFromFile();
11 | }
12 |
13 | // Rebuild the rollIndex map mapping rollNo -> vector index.
14 | // Time Complexity: O(n) average where n is the number of students, since we iterate over the vector
15 | // and insert each element into the hash map in O(1) average time.
16 | void AttendanceSystem::rebuildRollIndex() {
17 |     rollIndex.clear();
18 |     for (size_t i = 0; i < students.size(); ++i) {
19 |         rollIndex[students[i].getRollNo()] = static_cast<int>(i);
20 |     }
21 | }
22 |
23 | // Load students from file and build roll index map
24 | // Time Complexity: O(n) average to read and build the map.
25 | void AttendanceSystem::loadFromFile() {
26 |     FileManager::loadStudents(students);
27 |     rebuildRollIndex();
28 | }
29 |
30 | // Save current student data to file
31 | // Time Complexity: O(n) to serialize all elements and write to file.
32 | void AttendanceSystem::saveToFile() const {
33 |     FileManager::saveStudents(students);
34 | }
35 |
36 | // Get const reference to students
37 | // Time Complexity: O(1)
38 | const std::vector<Student>& AttendanceSystem::getStudents() const {
39 |     return students;
40 | }
41 |
42 | // Get mutable reference to students (useful for sorting)
43 | // Time Complexity: O(1)
44 | std::vector<Student>& AttendanceSystem::getStudentsMutable() {
45 |     return students;
46 | }
47 |
48 | // Add a new student
49 | // Time Complexity:
50 | //   - Checking duplicates: O(1) average / O(n) worst-case map lookup.
51 | //   - Pushing to vector: O(1) amortized.
52 | //   - Inserting into map: O(1) average / O(n) worst-case map insertion.
53 | //   - Saving to file: O(n) file write.
54 | //   - Total Complexity: O(n) due to saving to file. Without file I/O, it is O(1) average.
55 | bool AttendanceSystem::addStudent(const Student& s) {
56 |     if (rollIndex.find(s.getRollNo()) != rollIndex.end()) {
57 |         std::cerr << "[Error] Student with Roll No " << s.getRollNo() << " already exists." << std::endl;
58 |         return false;
59 |     }
60 |     students.push_back(s);
61 |     rollIndex[s.getRollNo()] = static_cast<int>(students.size() - 1);
62 |     saveToFile();
63 |     return true;
64 | }
65 |
66 | // Delete a student by roll number
67 | // Time Complexity:

```

```

68 | // - Lookup index in map: O(1) average.
69 | // - Erasing from vector: O(n) since elements after index must be shifted left.
70 | // - Rebuilding map index: O(n) average.
71 | // - Saving to file: O(n) file write.
72 | // - Total Complexity: O(n) because of shifting, rebuilding, and file writing.
73 | bool AttendanceSystem::deleteStudent(int rollNo) {
74 |     auto it = rollIndex.find(rollNo);
75 |     if (it == rollIndex.end()) {
76 |         std::cerr << "[Error] Student with Roll No " << rollNo << " not found." << std::endl;
77 |         return false;
78 |     }
79 |     int index = it->second;
80 |     students.erase(students.begin() + index);
81 |     rebuildRollIndex();
82 |     saveToFile();
83 |     return true;
84 | }
85 |
86 | // Update student details
87 | // Time Complexity:
88 | // - Lookup index in map: O(1) average.
89 | // - Updating attributes: O(1).
90 | // - Saving to file: O(n) file write.
91 | // - Total Complexity: O(n) because of saving to file. Without file I/O, it is O(1) average.
92 | bool AttendanceSystem::updateStudent(int rollNo, const std::string& newName, const std::string& newDept, int
newTotal, int newAttended) {
93 |     auto it = rollIndex.find(rollNo);
94 |     if (it == rollIndex.end()) {
95 |         std::cerr << "[Error] Student with Roll No " << rollNo << " not found." << std::endl;
96 |         return false;
97 |     }
98 |     int index = it->second;
99 |     students[index].setName(newName);
100 |     students[index].setDepartment(newDept);
101 |     students[index].setTotalClasses(newTotal);
102 |     students[index].setAttendedClasses(newAttended);
103 |     saveToFile();
104 |     return true;
105 | }
106 |
107 | // Helper: Find index of a student by Roll No
108 | // Time Complexity: O(1) average, O(n) worst-case lookup in unordered_map.
109 | int AttendanceSystem::findIndexByRollNo(int rollNo) const {
110 |     auto it = rollIndex.find(rollNo);
111 |     if (it != rollIndex.end()) {
112 |         return it->second;
113 |     }
114 |     return -1;
115 | }
116 |
117 | // Mark attendance session for a batch of roll numbers using a queue (FIFO).
118 | // Time Complexity: O(k + n) where k is the size of the batch, and n is the number of students.
119 | // We process each roll number in O(1) lookup time using the hash map, and then write to file once at the
end in O(n) time.
120 | // Without the single-write optimization, it would be O(k * n) due to repeated disk writes.
121 | void AttendanceSystem::markAttendanceSession(const std::vector<int>& rollNumbers) {
122 |     std::queue<int> q;
123 |     for (int roll : rollNumbers) {
124 |         q.push(roll);
125 |     }
126 |
127 |     bool modified = false;
128 |     while (!q.empty()) {
129 |         int roll = q.front();
130 |         q.pop();
131 |
132 |         int index = findIndexByRollNo(roll);
133 |         if (index == -1) {
134 |             std::cout << "[Warning] Student with Roll No " << roll << " not found in system. Skipping." <<
std::endl;
135 |             continue;
136 |         }
137 |

```

```

138 |         Student& s = students[index];
139 |         std::cout << "\nMarking attendance for: " << s.getName() << " (Roll No: " << roll << ")" <<
std::endl;
140 |         char choice;
141 |         while (true) {
142 |             std::cout << "Enter status (P for Present / A for Absent): ";
143 |             std::cin >> choice;
144 |             choice = std::tolower(choice);
145 |             if (choice == 'p' || choice == 'a') {
146 |                 break;
147 |             }
148 |             std::cout << "Invalid input. Please enter 'P' or 'A'." << std::endl;
149 |         }
150 |
151 |         std::string status = (choice == 'p') ? "Present" : "Absent";
152 |         int newTotal = s.getTotalClasses() + 1;
153 |         int newAttended = s.getAttendedClasses() + (choice == 'p' ? 1 : 0);
154 |         s.setTotalClasses(newTotal);
155 |         s.setAttendedClasses(newAttended);
156 |
157 |         // Push the action onto the undo stack
158 |         undoStack.push({roll, status});
159 |         modified = true;
160 |
161 |         std::cout << "Successfully marked " << status << " for " << s.getName() << "." << std::endl;
162 |     }
163 |
164 |     if (modified) {
165 |         saveToFile();
166 |     }
167 | }
168 |
169 | // Undo the last marked attendance action.
170 | // Time Complexity:
171 | // - Checking empty stack: O(1)
172 | // - Popping action: O(1)
173 | // - Map lookup to find index: O(1) average
174 | // - Mutating attendance counters: O(1)
175 | // - Saving file: O(n) file write.
176 | // - Total Complexity: O(n) due to saving to file. Without file I/O, it is O(1) average.
177 | bool AttendanceSystem::undoLastAttendance() {
178 |     if (undoStack.empty()) {
179 |         std::cout << "[Info] No attendance records to undo in the current session." << std::endl;
180 |         return false;
181 |     }
182 |
183 |     auto lastAction = undoStack.top();
184 |     undoStack.pop();
185 |
186 |     int roll = lastAction.first;
187 |     std::string status = lastAction.second;
188 |
189 |     int index = findIndexByRollNo(roll);
190 |     if (index == -1) {
191 |         std::cerr << "[Error] Could not undo: Student with Roll No " << roll << " no longer exists." <<
std::endl;
192 |         return false;
193 |     }
194 |
195 |     Student& s = students[index];
196 |
197 |     // Reverse the attendance marking action
198 |     int currentTotal = s.getTotalClasses();
199 |     int currentAttended = s.getAttendedClasses();
200 |
201 |     if (currentTotal > 0) {
202 |         s.setTotalClasses(currentTotal - 1);
203 |     }
204 |     if (status == "Present" && currentAttended > 0) {
205 |         s.setAttendedClasses(currentAttended - 1);
206 |     }
207 |
208 |     std::cout << "[Undo Success] Reversed \" " << status << "\" mark for student "

```

```

209         << s.getName() << " (Roll No: " << roll << ")." << std::endl;
210
211     saveToFile();
212     return true;
213 }
214
215 // Display formatted table of all registered students
216 // Time Complexity: O(n) where n is the number of students. We iterate and print each student report.
217 // Space Complexity: O(1) auxiliary.
218 void AttendanceSystem::displayAllStudents() const {
219     if (students.empty()) {
220         std::cout << "\n[Info] No students found in the database." << std::endl;
221         return;
222     }
223
224     std::cout << "\n" << std::string(105, '=') << "\n";
225     std::cout << std::left << std::setw(10) << "Roll No"
226         << std::setw(20) << "Name"
227         << std::setw(15) << "Department"
228         << std::setw(15) << "Total Classes"
229         << std::setw(15) << "Attended"
230         << std::setw(15) << "Attendance %"
231         << std::setw(15) << "Eligibility" << "\n";
232     std::cout << std::string(105, '-') << "\n";
233
234     for (const auto& student : students) {
235         student.printReport();
236     }
237     std::cout << std::string(105, '=') << "\n";
238     std::cout << "Total Students: " << students.size() << "\n\n";
239 }
240
241 // Generate report of students with attendance below 75% cutoff
242 // Time Complexity: O(n) where n is the number of students.
243 // Space Complexity: O(1) auxiliary.
244 void AttendanceSystem::lowAttendanceReport() const {
245     if (students.empty()) {
246         std::cout << "\n[Info] No students found in the database." << std::endl;
247         return;
248     }
249
250     bool foundAny = false;
251     std::cout << "\n" << std::string(60, '=') << "\n";
252     std::cout << "        LOW ATTENDANCE REPORT (< 75% Cutoff)\n";
253     std::cout << std::string(60, '-') << "\n";
254     std::cout << std::left << std::setw(10) << "Roll No"
255         << std::setw(25) << "Name"
256         << std::setw(15) << "Attendance %"
257         << std::setw(10) << "Deficit" << "\n";
258     std::cout << std::string(60, '-') << "\n";
259
260     for (const auto& student : students) {
261         double pct = student.getAttendancePercentage();
262         if (pct < 75.0) {
263             // Calculate how many more classes they needed to attend to reach 75%
264             // 0.75 * total = attended_needed
265             // deficit = attended_needed - attended
266             double needed = 0.75 * student.getTotalClasses();
267             double deficit = needed - student.getAttendedClasses();
268             int classDeficit = (deficit > 0) ? static_cast<int>(std::ceil(deficit)) : 0;
269
270             std::cout << std::left << std::setw(10) << student.getRollNo()
271                 << std::setw(25) << student.getName()
272                 << std::fixed << std::setprecision(2) << std::setw(15) << pct
273                 << std::setw(10) << classDeficit << "\n";
274             foundAny = true;
275         }
276     }
277     std::cout << std::string(60, '=') << "\n";
278
279     if (!foundAny) {
280         std::cout << "All students have satisfactory attendance (>= 75.00%)." << std::endl;
281     }

```

```
282 |     std::cout << std::endl;  
283 | }  
284 |  
285 |
```

FileManager.h

C++ Header — 80 lines

```

1 | #ifndef FILE_MANAGER_H
2 | #define FILE_MANAGER_H
3 |
4 | #include <vector>
5 | #include <string>
6 | #include <fstream>
7 | #include <iostream>
8 | #include <sstream>
9 | #include "Student.h"
10 |
11 | namespace FileManager {
12 |     const std::string FILENAME = "students.txt";
13 |
14 |     // Load students from students.txt into the vector.
15 |     // Time Complexity: O(n) where n is the number of student records in the file. We iterate once per line.
16 |     // Space Complexity: O(n) to store the students in the vector.
17 |     inline void loadStudents(std::vector<Student>& students) {
18 |         students.clear();
19 |         std::ifstream inFile(FILENAME);
20 |
21 |         // If the file does not exist, create it
22 |         if (!inFile) {
23 |             std::cout << "[Info] " << FILENAME << " not found. Creating a new file..." << std::endl;
24 |             std::ofstream outFile(FILENAME);
25 |             if (!outFile) {
26 |                 std::cerr << "[Error] Could not create file: " << FILENAME << std::endl;
27 |                 return;
28 |             }
29 |             outFile << "# rollNo name department totalClasses attendedClasses\n";
30 |             outFile.close();
31 |             return;
32 |         }
33 |
34 |         std::string line;
35 |         while (std::getline(inFile, line)) {
36 |             // Skip comments and empty lines
37 |             if (line.empty() || line[0] == '#') {
38 |                 continue;
39 |             }
40 |
41 |             std::stringstream ss(line);
42 |             int rollNo;
43 |             std::string name;
44 |             std::string department;
45 |             int totalClasses;
46 |             int attendedClasses;
47 |
48 |             // Read space-separated values
49 |             if (ss >> rollNo >> name >> department >> totalClasses >> attendedClasses) {
50 |                 students.push_back(Student(rollNo, name, department, totalClasses, attendedClasses));
51 |             }
52 |         }
53 |         inFile.close();
54 |         std::cout << "[Success] Loaded " << students.size() << " students from " << FILENAME << std::endl;
55 |     }
56 |
57 |     // Overwrite students.txt with current vector data.
58 |     // Time Complexity: O(n) where n is the number of students. We iterate once and write to disk.
59 |     // Space Complexity: O(1) auxiliary space.
60 |     inline void saveStudents(const std::vector<Student>& students) {
61 |         std::ofstream outFile(FILENAME);
62 |         if (!outFile) {
63 |             std::cerr << "[Error] Could not open file for writing: " << FILENAME << std::endl;
64 |             return;
65 |         }
66 |
67 |         outFile << "# rollNo name department totalClasses attendedClasses\n";

```

```
68 |         for (const auto& student : students) {
69 |             outFile << student.getRollNo() << " "
70 |                 << student.getName() << " "
71 |                 << student.getDepartment() << " "
72 |                 << student.getTotalClasses() << " "
73 |                 << student.getAttendedClasses() << "\n";
74 |         }
75 |         outFile.close();
76 |         std::cout << "[Success] Saved " << students.size() << " students to " << FILENAME << std::endl;
77 |     }
78 | }
79 |
80 | #endif // FILE_MANAGER_H
```

Search.h

C++ Header — 54 lines

```

1 | #ifndef SEARCH_H
2 | #define SEARCH_H
3 |
4 | #include <vector>
5 | #include "Student.h"
6 |
7 | namespace Search {
8 |
9 |     // Linear Search: Scan each student in the list one by one.
10 |    // Time Complexity: O(n) worst/average case, O(1) best case (if first element is target).
11 |    // Space Complexity: O(1) auxiliary.
12 |    inline int linearSearch(const std::vector<Student>& students, int rollNo) {
13 |        for (size_t i = 0; i < students.size(); ++i) {
14 |            if (students[i].getRollNo() == rollNo) {
15 |                return static_cast<int>(i); // Found, return index
16 |            }
17 |        }
18 |        return -1; // Not found
19 |    }
20 |
21 |    // Binary Search: Repeatedly divide the search interval in half.
22 |    // Time Complexity: O(log n) worst/average case, O(1) best case (if mid element is target).
23 |    // Space Complexity: O(1) auxiliary.
24 |    //
25 |    // WHY BINARY SEARCH REQUIRES SORTED DATA FIRST:
26 |    // Binary search relies on the ordering property of the dataset to make decisions.
27 |    // At each step, it compares the target roll number with the roll number of the middle student.
28 |    // - If the middle roll number is greater than target, sorting guarantees that all roll numbers to the
29 |    //   right of mid are also greater than target, allowing us to safely discard the right half.
30 |    // - If the middle roll number is smaller than target, sorting guarantees that all roll numbers to the
31 |    //   left of mid are also smaller than target, allowing us to safely discard the left half.
32 |    // Without sorted data, no such guarantees can be made, and discarding a half would risk throwing away
33 |    // the target element. Thus, unsorted data requires a full scan (Linear Search, O(n)).
34 |    inline int binarySearch(const std::vector<Student>& sortedStudents, int rollNo) {
35 |        int low = 0;
36 |        int high = static_cast<int>(sortedStudents.size()) - 1;
37 |
38 |        while (low <= high) {
39 |            int mid = low + (high - low) / 2; // Prevents potential integer overflow
40 |            int midRoll = sortedStudents[mid].getRollNo();
41 |
42 |            if (midRoll == rollNo) {
43 |                return mid; // Found, return index
44 |            } else if (midRoll < rollNo) {
45 |                low = mid + 1; // Discard left half
46 |            } else {
47 |                high = mid - 1; // Discard right half
48 |            }
49 |        }
50 |        return -1; // Not found
51 |    }
52 | }
53 |
54 | #endif // SEARCH_H

```

MergeSort.h

C++ Header — 139 lines

```

1 | #ifndef MERGE_SORT_H
2 | #define MERGE_SORT_H
3 |
4 | #include <vector>
5 | #include <string>
6 | #include <algorithm>
7 | #include "Student.h"
8 |
9 | // Enum to support multiple sorting criteria
10 | enum class SortCriteria {
11 |     ROLL_NO,
12 |     NAME,
13 |     ATTENDANCE_PCT
14 | };
15 |
16 | namespace Sort {
17 |
18 |     // Helper: Compare two students based on the specified criteria
19 |     inline bool compareStudents(const Student& a, const Student& b, SortCriteria criteria) {
20 |         if (criteria == SortCriteria::ROLL_NO) {
21 |             return a.getRollNo() < b.getRollNo();
22 |         } else if (criteria == SortCriteria::NAME) {
23 |             return a.getName() < b.getName();
24 |         } else { // SortCriteria::ATTENDANCE_PCT
25 |             return a.getAttendancePercentage() < b.getAttendancePercentage();
26 |         }
27 |     }
28 |
29 |     // Bubble Sort: Iteratively compares adjacent students and swaps them if they are in the wrong order.
30 |     // Time Complexity: O(n^2) worst/average case, O(n) best case (if already sorted).
31 |     // Space Complexity: O(1) auxiliary (In-place sort).
32 |     inline void bubbleSort(std::vector<Student>& students, SortCriteria criteria) {
33 |         int n = static_cast<int>(students.size());
34 |         for (int i = 0; i < n - 1; ++i) {
35 |             bool swapped = false;
36 |             for (int j = 0; j < n - i - 1; ++j) {
37 |                 // If students[j+1] is smaller than students[j], swap them
38 |                 if (compareStudents(students[j + 1], students[j], criteria)) {
39 |                     std::swap(students[j], students[j + 1]);
40 |                     swapped = true;
41 |                 }
42 |             }
43 |             // Optimization: If no two elements were swapped by inner loop, then list is sorted.
44 |             if (!swapped) {
45 |                 break;
46 |             }
47 |         }
48 |     }
49 |
50 |     // Helper: Merges two sorted subarrays of students: students[left..mid] and students[mid+1..right].
51 |     // Time Complexity: O(n) where n is the number of elements being merged.
52 |     // Space Complexity: O(n) auxiliary.
53 |     inline void merge(std::vector<Student>& students, int left, int mid, int right, SortCriteria criteria) {
54 |         int n1 = mid - left + 1;
55 |         int n2 = right - left - n1 + 1; // equivalent to right - mid
56 |
57 |         // Create temporary vectors to hold the halves
58 |         std::vector<Student> L(n1);
59 |         std::vector<Student> R(n2);
60 |
61 |         // Copy data to temporary vectors L[] and R[]
62 |         for (int i = 0; i < n1; ++i) {
63 |             L[i] = students[left + i];
64 |         }
65 |         for (int j = 0; j < n2; ++j) {
66 |             R[j] = students[mid + 1 + j];
67 |         }

```

```

68 |
69 | // Merge the temporary vectors back into students[left..right]
70 | int i = 0; // Initial index of first subarray (L)
71 | int j = 0; // Initial index of second subarray (R)
72 | int k = left; // Initial index of merged subarray
73 |
74 | while (i < n1 && j < n2) {
75 |     // compare L[i] and R[j]; standard comparison to keep stable sort
76 |     if (compareStudents(L[i], R[j], criteria) ||
77 |         (!compareStudents(R[j], L[i], criteria) && L[i].getRollNo() <= R[j].getRollNo())) {
78 |         students[k] = L[i];
79 |         i++;
80 |     } else {
81 |         students[k] = R[j];
82 |         j++;
83 |     }
84 |     k++;
85 | }
86 |
87 | // Copy any remaining elements of L[]
88 | while (i < n1) {
89 |     students[k] = L[i];
90 |     i++;
91 |     k++;
92 | }
93 |
94 | // Copy any remaining elements of R[]
95 | while (j < n2) {
96 |     students[k] = R[j];
97 |     j++;
98 |     k++;
99 | }
100 | }
101 |
102 | // Merge Sort: A recursive divide-and-conquer algorithm.
103 | // Time Complexity: O(n log n) in all cases (worst, average, best).
104 | // Space Complexity: O(n) auxiliary (due to temporary sub-vectors in merge step).
105 | //
106 | // EXPLANATION OF RECURSION IN MERGE SORT:
107 | // Merge Sort divides the problem into subproblems, solves them recursively, and combines the results.
108 | //
109 | // 1. DIVIDE STEP: Find the midpoint index 'mid = left + (right - left) / 2' to split the array
110 | //    into two halves: [left...mid] and [mid+1...right].
111 | //
112 | // 2. CONQUER STEP (Recursive Calls):
113 | //    - Recursively sort the left half: mergeSort(students, left, mid, criteria)
114 | //    - Recursively sort the right half: mergeSort(students, mid + 1, right, criteria)
115 | //    The base case for this recursion is when 'left >= right' (i.e. subarray has size 0 or 1,
116 | //    which is already sorted).
117 | //
118 | // 3. COMBINE STEP (Merge):
119 | //    - Call merge(students, left, mid, right, criteria) to take the two sorted halves and
120 | //    combine them back into a single sorted range.
121 | inline void mergeSort(std::vector<Student>& students, int left, int right, SortCriteria criteria) {
122 |     // Base case: If subarray has 1 or 0 elements, it is already sorted.
123 |     if (left >= right) {
124 |         return;
125 |     }
126 |
127 |     // 1. DIVIDE
128 |     int mid = left + (right - left) / 2;
129 |
130 |     // 2. CONQUER (Recursive division of left and right halves)
131 |     mergeSort(students, left, mid, criteria);
132 |     mergeSort(students, mid + 1, right, criteria);
133 |
134 |     // 3. COMBINE (Merge the sorted halves back together)
135 |     merge(students, left, mid, right, criteria);
136 | }
137 | }
138 |
139 | #endif // MERGE_SORT_H

```

main.cpp

C++ Source — 313 lines

```

1 | #include <iostream>
2 | #include <vector>
3 | #include <string>
4 | #include <limits>
5 | #include <algorithm>
6 | #include <iomanip>
7 | #include "AttendanceSystem.h"
8 | #include "Search.h"
9 | #include "MergeSort.h"
10 |
11 | // Helper function to safely read an integer from console input
12 | int readInteger(const std::string& prompt) {
13 |     int value;
14 |     while (true) {
15 |         std::cout << prompt;
16 |         if (std::cin >> value) {
17 |             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // clear line buffer
18 |             return value;
19 |         }
20 |         std::cout << "[Error] Invalid input. Please enter a valid integer.\n";
21 |         std::cin.clear();
22 |         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
23 |     }
24 | }
25 |
26 | // Helper function to safely read a string from console input (handles single-word input)
27 | std::string readString(const std::string& prompt) {
28 |     std::string value;
29 |     while (true) {
30 |         std::cout << prompt;
31 |         if (std::cin >> value) {
32 |             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // clear line buffer
33 |             return value;
34 |         }
35 |         std::cout << "[Error] Invalid input. Please enter a string.\n";
36 |         std::cin.clear();
37 |         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
38 |     }
39 | }
40 |
41 | // Display high-level database stats for Option 7
42 | void displayAttendanceStats(const AttendanceSystem& sys) {
43 |     const auto& students = sys.getStudents();
44 |     if (students.empty()) {
45 |         std::cout << "\n[Info] No students registered in the system.\n";
46 |         return;
47 |     }
48 |
49 |     double totalPct = 0;
50 |     int eligibleCount = 0;
51 |     int totalSt = static_cast<int>(students.size());
52 |
53 |     for (const auto& s : students) {
54 |         double pct = s.getAttendancePercentage();
55 |         totalPct += pct;
56 |         if (pct >= 75.0) {
57 |             eligibleCount++;
58 |         }
59 |     }
60 |
61 |     double averagePct = totalPct / totalSt;
62 |
63 |     std::cout << "\n" << std::string(50, '=') << "\n";
64 |     std::cout << "                SYSTEM ATTENDANCE STATISTICS\n";
65 |     std::cout << std::string(50, '-') << "\n";
66 |     std::cout << "Total Enrolled Students : " << totalSt << "\n";
67 |     std::cout << "Average Attendance Rate : " << std::fixed << std::setprecision(2) << averagePct << "%\n";

```

```

68 |     std::cout << "Eligible Students (>=75%): " << eligibleCount << " (" << (eligibleCount * 100.0 / totalSt)
<< "%)\n";
69 |     std::cout << "Deficient Students (<75%): " << (totalSt - eligibleCount) << " (" << ((totalSt -
eligibleCount) * 100.0 / totalSt) << "%)\n";
70 |     std::cout << std::string(50, '=') << "\n\n";
71 |
72 |     // Also display the table
73 |     sys.displayAllStudents();
74 | }
75 |
76 | int main() {
77 |     AttendanceSystem sys; // Automatically calls loadFromFile() on startup
78 |
79 |     int choice = 0;
80 |     do {
81 |         std::cout << "=====\n";
82 |         std::cout << "          STUDENT ATTENDANCE MANAGEMENT SYSTEM          \n";
83 |         std::cout << "=====\n";
84 |         std::cout << " 1. Add Student\n";
85 |         std::cout << " 2. Delete Student\n";
86 |         std::cout << " 3. Update Student\n";
87 |         std::cout << " 4. Search Student\n";
88 |         std::cout << " 5. Display Students\n";
89 |         std::cout << " 6. Mark Attendance (Batch Session)\n";
90 |         std::cout << " 7. Attendance Report & Stats\n";
91 |         std::cout << " 8. Low Attendance List (<75%)\n";
92 |         std::cout << " 9. Sort Students by Roll Number\n";
93 |         std::cout << "10. Sort Students by Name\n";
94 |         std::cout << "11. Sort Students by Attendance %\n";
95 |         std::cout << "12. Save Data\n";
96 |         std::cout << "13. Undo Last Attendance Action\n";
97 |         std::cout << "14. Exit (Auto-saves)\n";
98 |         std::cout << "-----\n";
99 |
100 |         choice = readInteger("Enter choice (1-14): ");
101 |
102 |         switch (choice) {
103 |             case 1: { // Add Student
104 |                 std::cout << "\n--- Add New Student ---\n";
105 |                 int rollNo = readInteger("Enter Roll No: ");
106 |                 std::string name = readString("Enter Name (use underscores for spaces, e.g. John_Doe): ");
107 |                 std::string dept = readString("Enter Department (e.g. CSE): ");
108 |                 int totalClasses = readInteger("Enter Total Classes Held: ");
109 |                 int attendedClasses = readInteger("Enter Classes Attended: ");
110 |
111 |                 if (rollNo <= 0 || totalClasses < 0 || attendedClasses < 0) {
112 |                     std::cout << "[Error] Input values must be non-negative, and Roll No must be
positive.\n\n";
113 |                 } else if (attendedClasses > totalClasses) {
114 |                     std::cout << "[Error] Attended classes cannot exceed total classes.\n\n";
115 |                 } else {
116 |                     Student newStudent(rollNo, name, dept, totalClasses, attendedClasses);
117 |                     if (sys.addStudent(newStudent)) {
118 |                         std::cout << "[Success] Student added successfully!\n\n";
119 |                     }
120 |                 }
121 |                 break;
122 |             }
123 |             case 2: { // Delete Student
124 |                 std::cout << "\n--- Delete Student ---\n";
125 |                 int rollNo = readInteger("Enter Roll No of student to delete: ");
126 |                 if (sys.deleteStudent(rollNo)) {
127 |                     std::cout << "[Success] Student removed successfully.\n\n";
128 |                 }
129 |                 break;
130 |             }
131 |             case 3: { // Update Student
132 |                 std::cout << "\n--- Update Student Details ---\n";
133 |                 int rollNo = readInteger("Enter Roll No of student to update: ");
134 |                 int index = sys.findIndexByRollNo(rollNo);
135 |                 if (index == -1) {
136 |                     std::cout << "[Error] Student with Roll No " << rollNo << " not found.\n\n";
137 |                 } else {

```

```

138 |         std::string name = readString("Enter New Name (use underscores for spaces): ");
139 |         std::string dept = readString("Enter New Department: ");
140 |         int totalClasses = readInteger("Enter New Total Classes Held: ");
141 |         int attendedClasses = readInteger("Enter New Classes Attended: ");
142 |
143 |         if (totalClasses < 0 || attendedClasses < 0) {
144 |             std::cout << "[Error] Class counts cannot be negative.\n\n";
145 |         } else if (attendedClasses > totalClasses) {
146 |             std::cout << "[Error] Attended classes cannot exceed total classes.\n\n";
147 |         } else {
148 |             if (sys.updateStudent(rollNo, name, dept, totalClasses, attendedClasses)) {
149 |                 std::cout << "[Success] Student details updated successfully!\n\n";
150 |             }
151 |         }
152 |     }
153 |     break;
154 | }
155 | case 4: { // Search Student
156 |     std::cout << "\n--- Search Student ---\n";
157 |     int rollNo = readInteger("Enter Roll No to search: ");
158 |     std::cout << "Choose search algorithm:\n";
159 |     std::cout << " 1. Linear Search (O(n) - Works on unsorted list)\n";
160 |     std::cout << " 2. Binary Search (O(log n) - Assumes list is sorted by Roll No)\n";
161 |     int alg = readInteger("Enter algorithm choice (1-2): ");
162 |
163 |     int foundIndex = -1;
164 |     const auto& list = sys.getStudents();
165 |
166 |     if (alg == 1) {
167 |         foundIndex = Search::linearSearch(list, rollNo);
168 |     } else if (alg == 2) {
169 |         // Quick validation to warn user if list is not sorted by roll no
170 |         bool isSorted = true;
171 |         for (size_t i = 1; i < list.size(); ++i) {
172 |             if (list[i].getRollNo() < list[i - 1].getRollNo()) {
173 |                 isSorted = false;
174 |                 break;
175 |             }
176 |         }
177 |         if (!isSorted) {
178 |             std::cout << "[Warning] Dataset is NOT currently sorted by Roll No. Binary search
may fail.\n";
179 |
180 |             std::cout << "Attempting to search anyway...\n";
181 |             foundIndex = Search::binarySearch(list, rollNo);
182 |         } else {
183 |             std::cout << "[Error] Invalid algorithm choice. Cancelling search.\n\n";
184 |             break;
185 |         }
186 |
187 |         if (foundIndex != -1) {
188 |             std::cout << "\n[Success] Student found at index " << foundIndex << "!\n";
189 |             std::cout << std::string(105, '-') << "\n";
190 |             std::cout << std::left << std::setw(10) << "Roll No"
191 |                 << std::setw(20) << "Name"
192 |                 << std::setw(15) << "Department"
193 |                 << std::setw(15) << "Total Classes"
194 |                 << std::setw(15) << "Attended"
195 |                 << std::setw(15) << "Attendance %"
196 |                 << std::setw(15) << "Eligibility" << "\n";
197 |             std::cout << std::string(105, '-') << "\n";
198 |             list[foundIndex].printReport();
199 |             std::cout << std::string(105, '-') << "\n\n";
200 |         } else {
201 |             std::cout << "[Result] Student with Roll No " << rollNo << " not found.\n\n";
202 |         }
203 |         break;
204 |     }
205 | case 5: { // Display Students
206 |     std::cout << "\n--- Registered Students ---";
207 |     sys.displayAllStudents();
208 |     break;
209 | }

```

```

210 | case 6: { // Mark Attendance
211 |     std::cout << "\n--- Mark Attendance Session ---\n";
212 |     int count = readInteger("Enter number of students to mark attendance for: ");
213 |     if (count <= 0) {
214 |         std::cout << "[Error] Count must be greater than zero.\n\n";
215 |         break;
216 |     }
217 |     std::vector<int> rolls;
218 |     for (int i = 0; i < count; ++i) {
219 |         rolls.push_back(readInteger("Enter Roll No for student #" + std::to_string(i + 1) + ":
220 |     });
221 |     }
222 |     sys.markAttendanceSession(rolls);
223 |     std::cout << "\n[Session Finished] Attendance batch processed.\n\n";
224 |     break;
225 | }
226 | case 7: { // Attendance Report & Stats
227 |     displayAttendanceStats(sys);
228 |     break;
229 | }
230 | case 8: { // Low Attendance List
231 |     sys.lowAttendanceReport();
232 |     break;
233 | }
234 | case 9: { // Sort by Roll
235 |     std::cout << "\n--- Sort Students by Roll Number ---\n";
236 |     std::cout << "Choose algorithm:\n 1. Bubble Sort (O(n^2))\n 2. Merge Sort (O(n log n))\n";
237 |     int alg = readInteger("Enter algorithm choice (1-2): ");
238 |
239 |     auto& mutableList = sys.getStudentsMutable();
240 |     if (alg == 1) {
241 |         Sort::bubbleSort(mutableList, SortCriteria::ROLL_NO);
242 |         std::cout << "[Success] Sorted using Bubble Sort.\n\n";
243 |     } else if (alg == 2) {
244 |         if (!mutableList.empty()) {
245 |             Sort::mergeSort(mutableList, 0, static_cast<int>(mutableList.size()) - 1,
SortCriteria::ROLL_NO);
246 |         }
247 |         std::cout << "[Success] Sorted using Merge Sort.\n\n";
248 |     } else {
249 |         std::cout << "[Error] Invalid algorithm choice. Sorting aborted.\n\n";
250 |     }
251 |     break;
252 | }
253 | case 10: { // Sort by Name
254 |     std::cout << "\n--- Sort Students by Name ---\n";
255 |     std::cout << "Choose algorithm:\n 1. Bubble Sort (O(n^2))\n 2. Merge Sort (O(n log n))\n";
256 |     int alg = readInteger("Enter algorithm choice (1-2): ");
257 |
258 |     auto& mutableList = sys.getStudentsMutable();
259 |     if (alg == 1) {
260 |         Sort::bubbleSort(mutableList, SortCriteria::NAME);
261 |         std::cout << "[Success] Sorted using Bubble Sort.\n\n";
262 |     } else if (alg == 2) {
263 |         if (!mutableList.empty()) {
264 |             Sort::mergeSort(mutableList, 0, static_cast<int>(mutableList.size()) - 1,
SortCriteria::NAME);
265 |         }
266 |         std::cout << "[Success] Sorted using Merge Sort.\n\n";
267 |     } else {
268 |         std::cout << "[Error] Invalid algorithm choice. Sorting aborted.\n\n";
269 |     }
270 |     break;
271 | }
272 | case 11: { // Sort by Attendance %
273 |     std::cout << "\n--- Sort Students by Attendance Percentage ---\n";
274 |     std::cout << "Choose algorithm:\n 1. Bubble Sort (O(n^2))\n 2. Merge Sort (O(n log n))\n";
275 |     int alg = readInteger("Enter algorithm choice (1-2): ");
276 |
277 |     auto& mutableList = sys.getStudentsMutable();
278 |     if (alg == 1) {
279 |         Sort::bubbleSort(mutableList, SortCriteria::ATTENDANCE_PCT);
280 |         std::cout << "[Success] Sorted using Bubble Sort.\n\n";

```

```

280 |         } else if (alg == 2) {
281 |             if (!mutableList.empty()) {
282 |                 Sort::mergeSort(mutableList, 0, static_cast<int>(mutableList.size()) - 1,
SortCriteria::ATTENDANCE_PCT);
283 |             }
284 |             std::cout << "[Success] Sorted using Merge Sort.\n\n";
285 |         } else {
286 |             std::cout << "[Error] Invalid algorithm choice. Sorting aborted.\n\n";
287 |         }
288 |         break;
289 |     }
290 |     case 12: { // Save Data
291 |         sys.saveToFile();
292 |         std::cout << "\n";
293 |         break;
294 |     }
295 |     case 13: { // Undo Last Attendance
296 |         std::cout << "\n--- Undo Last Attendance Action ---\n";
297 |         sys.undoLastAttendance();
298 |         std::cout << "\n";
299 |         break;
300 |     }
301 |     case 14: { // Exit
302 |         sys.saveToFile();
303 |         std::cout << "\nExiting Student Attendance Management System. Goodbye!\n";
304 |         break;
305 |     }
306 |     default:
307 |         std::cout << "[Error] Invalid choice. Please choose between 1 and 14.\n\n";
308 |         break;
309 |     }
310 | } while (choice != 14);
311 |
312 | return 0;
313 | }

```

index.html

HTML — 537 lines

```

1 | <!DOCTYPE html>
2 | <html lang="en">
3 | <head>
4 |     <meta charset="UTF-8">
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 |     <title>SAMS | Student Attendance Management</title>
7 |     <!-- Google Fonts -->
8 |     <link rel="preconnect" href="https://fonts.googleapis.com">
9 |     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 |     <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&family=Plus+Jakarta+Sans:wght@3
00;400;500;600;700;800&display=swap" rel="stylesheet">
11 |     <link rel="stylesheet" href="style.css">
12 | </head>
13 | <body>
14 |     <!-- Interactive Background Canvas -->
15 |     <canvas id="particle-canvas"></canvas>
16 |
17 |     <div class="app-container">
18 |         <!-- Sidebar Navigation -->
19 |         <aside class="sidebar glass-panel">
20 |             <div class="sidebar-header">
21 |                 <div class="logo-icon">
22 |                     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><path d="M22 10v6M2 10l10-5 10 5-10 5z"/><path
d="M6 12v5c0 2 2 3 6 3s6-1 6-3v-5"/></svg>
23 |                     </div>
24 |                     <h2>AMS Portal</h2>
25 |                 </div>
26 |
27 |                 <nav class="sidebar-menu">
28 |                     <button class="menu-item active" data-section="dashboard" id="btn-nav-dashboard">
29 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><rect x="3" y="3" width="7" height="9"/><rect
x="14" y="3" width="7" height="5"/><rect x="14" y="12" width="7" height="9"/><rect x="3" y="16" width="7"
height="5"/></svg>
30 |                         <span>Dashboard</span>
31 |                     </button>
32 |                     <button class="menu-item" data-section="add-student" id="btn-nav-add">
33 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M16 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4
4v2"/><circle cx="8.5" cy="7" r="4"/><line x1="20" y1="8" x2="20" y2="14"/><line x1="17" y1="11" x2="23"
y2="11"/></svg>
34 |                         <span>Add Student</span>
35 |                     </button>
36 |                     <button class="menu-item" data-section="search" id="btn-nav-search">
37 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11" r="8"/><line x1="21"
y1="21" x2="16.65" y2="16.65"/></svg>
38 |                         <span>Search</span>
39 |                     </button>
40 |                     <button class="menu-item" data-section="attendance" id="btn-nav-attendance">
41 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M9 11l3 3L22 4"/><path d="M21 12v7a2 2 0
0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"/></svg>
42 |                         <span>Mark Attendance</span>
43 |                     </button>
44 |                     <button class="menu-item" data-section="sorting" id="btn-nav-sorting">
45 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><line x1="18" y1="20" x2="18" y2="10"/><line
x1="12" y1="20" x2="12" y2="4"/><line x1="6" y1="20" x2="6" y2="14"/></svg>
46 |                         <span>Sorting Algorithms</span>
47 |                     </button>
48 |                     <button class="menu-item" data-section="reports" id="btn-nav-reports">
49 |                         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2

```

```

2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16"
y1="17" x2="8" y2="17"/><polyline points="10 9 9 8 9"/></svg>
50 |         <span>Reports & Charts</span>
51 |     </button>
52 |     <button class="menu-item" data-section="about" id="btn-nav-about">
53 |         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="10"/><line x1="12"
y1="16" x2="12" y2="12"/><line x1="12" y1="8" x2="12.01" y2="8"/></svg>
54 |         <span>About & DSA Info</span>
55 |     </button>
56 | </nav>
57 |
58 |     <div class="sidebar-footer">
59 |         <p>DSA Summer Training</p>
60 |         <span class="version">v3.5 Quantum</span>
61 |     </div>
62 | </aside>
63 |
64 | <!-- Main Content Area -->
65 | <main class="main-content">
66 |     <!-- Header bar -->
67 |     <header class="content-header glass-panel">
68 |         <div class="header-title">
69 |             <h1 id="page-title">Dashboard</h1>
70 |             <p id="page-subtitle">Welcome to the Attendance Management System dashboard</p>
71 |         </div>
72 |         <div class="header-actions">
73 |             <div class="theme-pill">
74 |                 <span class="pill-badge">Active Session</span>
75 |             </div>
76 |         </div>
77 |     </header>
78 |
79 |     <div class="content-body">
80 |         <!-- Section: Dashboard -->
81 |         <section id="sec-dashboard" class="content-section active">
82 |             <div class="welcome-card glass-panel">
83 |                 <div class="welcome-text">
84 |                     <h3>Ready to organize your lectures?</h3>
85 |                     <p>Manage courses, batch-mark student presence using queue processing, verify
indices via binary search, and visualize sorting algorithms in real time.</p>
86 |                 </div>
87 |                 <div class="welcome-illustration">
88 |                     <div class="circle-glow"></div>
89 |                 </div>
90 |             </div>
91 |
92 |             <div class="stats-grid">
93 |                 <div class="stat-card glass-panel" data-glow>
94 |                     <div class="stat-icon">
95 |                         <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
primary-light)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M17 21v-2a4 4 0 0 0-4-
4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0
1 0 7.75"/></svg>
96 |                     </div>
97 |                     <div class="stat-details">
98 |                         <span class="stat-label">Total Students</span>
99 |                         <h4 class="stat-value" id="dash-stat-total">0</h4>
100 |                     </div>
101 |                 </div>
102 |                 <div class="stat-card glass-panel" data-glow>
103 |                     <div class="stat-icon">
104 |                         <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
success)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><polyline points="12 6 12 16 14"/></svg>
105 |                     </div>
106 |                     <div class="stat-details">
107 |                         <span class="stat-label">Average Attendance</span>
108 |                         <h4 class="stat-value" id="dash-stat-avg">0.0%</h4>
109 |                     </div>
110 |                 </div>
111 |                 <div class="stat-card glass-panel" data-glow>
112 |                     <div class="stat-icon">

```

```

113 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
danger)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polygon points="7.86 2 16.14 2 22 7.86
22 16.14 16.14 22 7.86 22 2 16.14 2 7.86 7.86 2"/><line x1="12" y1="8" x2="12" y2="12"/><line x1="12" y1="16"
x2="12.01" y2="16"/></svg>
114 |                 </div>
115 |                 <div class="stat-details">
116 |                     <span class="stat-label">Deficient Students</span>
117 |                     <h4 class="stat-value" id="dash-stat-low">0</h4>
118 |                 </div>
119 |             </div>
120 |         </div>
121 |     </section>
122 |
123 |     <!-- Section: Add Student -->
124 |     <section id="sec-add-student" class="content-section">
125 |         <div class="form-table-split">
126 |             <div class="card card-form glass-panel" data-glow>
127 |                 <div class="card-header-accent">
128 |                     <h3 id="form-title">Register New Student</h3>
129 |                     <p id="form-desc">Insert details to register a student record into local
memory store.</p>
130 |                 </div>
131 |                 <div class="card-body">
132 |                     <form id="form-student" class="student-form">
133 |                         <div class="form-group-row">
134 |                             <div class="form-group">
135 |                                 <label for="student-roll">Roll Number</label>
136 |                                 <input type="number" id="student-roll" placeholder="e.g. 101"
required min="1">
137 |                             </div>
138 |                             <div class="form-group">
139 |                                 <label for="student-dept">Department</label>
140 |                                 <input type="text" id="student-dept" placeholder="e.g. CSE"
required>
141 |                             </div>
142 |                         </div>
143 |                         <div class="form-group">
144 |                             <label for="student-name">Full Name</label>
145 |                             <input type="text" id="student-name" placeholder="e.g. John Doe"
required>
146 |                         </div>
147 |                         <div class="form-group-row">
148 |                             <div class="form-group">
149 |                                 <label for="student-total">Total Classes</label>
150 |                                 <input type="number" id="student-total" placeholder="e.g. 40"
required min="0">
151 |                             </div>
152 |                             <div class="form-group">
153 |                                 <label for="student-attended">Attended Classes</label>
154 |                                 <input type="number" id="student-attended" placeholder="e.g. 35"
required min="0">
155 |                             </div>
156 |                         </div>
157 |
158 |                     <div class="error-banner" id="form-error-banner" style="display:
none;"></div>
159 |                     <div class="success-banner" id="form-success-banner" style="display:
none;"></div>
160 |
161 |                     <div class="form-actions">
162 |                         <button type="submit" id="btn-submit-student" class="btn btn-
primary">Add Student</button>
163 |                         <button type="button" id="btn-cancel-edit" class="btn btn-secondary"
style="display: none;">Cancel</button>
164 |                     </div>
165 |                 </form>
166 |             </div>
167 |         </div>
168 |
169 |         <div class="card directory-card glass-panel" data-glow>
170 |             <div class="card-header-accent" style="display: flex; justify-content: space-
between; align-items: center;">
171 |                 <div>

```

```

172 |         <h3>Student Directory</h3>
173 |         <p>Manage and review all registered students in the system.</p>
174 |     </div>
175 |     <button id="btn-export-csv" class="btn btn-secondary" style="padding: 0.5rem
1rem; font-size: 0.85rem; display: flex; align-items: center; gap: 0.5rem;">
176 |         <svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><path d="M21 15v4a2 2 0 0
1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
177 |         Export CSV
178 |     </button>
179 | </div>
180 | <div class="card-body scroll-x">
181 |     <table class="table" id="table-directory">
182 |         <thead>
183 |             <tr>
184 |                 <th>Roll No</th>
185 |                 <th>Name</th>
186 |                 <th>Dept</th>
187 |                 <th>Total</th>
188 |                 <th>Attended</th>
189 |                 <th>%</th>
190 |                 <th class="text-right">Actions</th>
191 |             </tr>
192 |         </thead>
193 |         <tbody id="directory-tbody">
194 |             <!-- Rendered dynamically -->
195 |         </tbody>
196 |     </table>
197 | </div>
198 | </div>
199 | </div>
200 | </section>
201 |
202 | <!-- Section: Search -->
203 | <section id="sec-search" class="content-section">
204 |     <div class="search-container-grid">
205 |         <div class="card card-search-controls glass-panel" data-glow>
206 |             <div class="card-header-accent">
207 |                 <h3>Lookup Configuration</h3>
208 |                 <p>Set target Roll No and select the lookup algorithm.</p>
209 |             </div>
210 |             <div class="card-body">
211 |                 <form id="form-search" class="student-form">
212 |                     <div class="form-group">
213 |                         <label for="search-roll-input">Roll Number</label>
214 |                         <input type="number" id="search-roll-input" placeholder="e.g. 101"
required min="1">
215 |                     </div>
216 |                     <div class="form-group">
217 |                         <label>Search Method</label>
218 |                         <div class="radio-pill-group">
219 |                             <label class="radio-pill">
220 |                                 <input type="radio" name="search-method" value="linear"
checked>
221 |                                 <span>Linear Search</span>
222 |                             </label>
223 |                             <label class="radio-pill">
224 |                                 <input type="radio" name="search-method" value="binary">
225 |                                 <span>Binary Search</span>
226 |                             </label>
227 |                         </div>
228 |                     </div>
229 |                     <button type="submit" id="btn-search-submit" class="btn btn-primary"
style="margin-top: 1rem; width: 100%;">
230 |                         <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
231 |                         Execute Search
232 |                     </button>
233 |                 </form>
234 |             </div>
235 |         </div>
236 |     </div>

```

```

237 |                 <div class="card card-search-results glass-panel" data-glow>
238 |                     <div class="card-header-accent" style="display: flex; justify-content: space-
between; align-items: center;">
239 |                         <div>
240 |                             <h3>Search Outcome & Benchmarks</h3>
241 |                             <p>Real-time analytics comparing  $O(N)$  scan against  $O(\log N)$ 
division.</p>
242 |                         </div>
243 |                         <div id="search-timer-badge" class="timer-badge" style="display: none;">
244 |                             <svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><polyline points="12 6 12 12 16 14"/></svg>
245 |                             <span id="search-time-taken">0.00 ms</span>
246 |                         </div>
247 |                     </div>
248 |                     <div class="card-body" id="search-results-area">
249 |                         <div style="display: flex; flex-direction: column; align-items: center;
justify-content: center; height: 100%; color: var(--text-muted); opacity: 0.5;">
250 |                             <svg width="64" height="64" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round" style="margin-bottom:
1rem;"><circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/><line x1="8" y1="11" x2="14"
y2="11"/></svg>
251 |                             <p>Enter a student roll number and run search to see results.</p>
252 |                         </div>
253 |                     </div>
254 |                 </div>
255 |             </div>
256 |         </section>
257 |
258 |         <!-- Section: Attendance -->
259 |         <section id="sec-attendance" class="content-section">
260 |             <div class="attendance-dashboard">
261 |                 <div class="card card-attendance-init glass-panel" data-glow>
262 |                     <div class="card-header-accent">
263 |                         <h3>Initiate Marking Session</h3>
264 |                         <p>Initialize a FIFO marking queue by adding roll numbers.</p>
265 |                     </div>
266 |                     <div class="card-body">
267 |                         <form id="form-attendance-init" class="student-form">
268 |                             <div class="form-group">
269 |                                 <label for="attendance-rolls-input">Roll Numbers (comma/space
separated)</label>
270 |                                 <textarea id="attendance-rolls-input" rows="3" placeholder="e.g.
101, 102, 105" required></textarea>
271 |                             </div>
272 |                             <button type="submit" id="btn-start-attendance" class="btn btn-primary"
style="width: 100%;">
273 |                                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polygon points="5 3 19 12 5
21 5 3"/></svg>
274 |                                 Launch Session
275 |                             </button>
276 |                         </form>
277 |                         <div id="queue-preview-box" style="display: none; margin-top: 1.5rem;">
278 |                             <span style="font-size: 0.8rem; font-weight: 600; color: var(--text-
muted); text-transform: uppercase; display: block; margin-bottom: 0.5rem;">Pending Queue (FIFO)</span>
279 |                             <div id="queue-preview-pills" style="display: flex; flex-wrap: wrap;
gap: 0.5rem;">
280 |                                 <!-- Rendered dynamically -->
281 |                             </div>
282 |                         </div>
283 |                     </div>
284 |                 </div>
285 |
286 |                 <div class="card card-attendance-marking glass-panel" data-glow>
287 |                     <div class="card-header-accent">
288 |                         <h3>Active Marking Queue</h3>
289 |                         <p>Mark students Present or Absent in sequential FIFO order.</p>
290 |                     </div>
291 |                     <div class="card-body" id="attendance-marking-area" style="min-height: 250px;
display: flex; align-items: center; justify-content: center;">
292 |                         <div style="display: flex; flex-direction: column; align-items: center;
justify-content: center; gap: 1rem; color: var(--text-muted); opacity: 0.5; text-align: center;">

```

```

293 |         <svg width="64" height="64" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"><path d="M14.7 6.3a1 1 0 0
0 0 1.411.6 1.6a1 1 0 0 0 1.4 013.77-3.77a1 1 0 0 1 1.41 01.02.02a1 1 0 0 1 0 1.411-4.48 4.48a1 1 0 0 1-1.41 01-
2.32-2.32a1 1 0 0 0-1.41 01-.02.02a1 1 0 0 0 0 1.4113.03 3.03a1 1 0 0 0 1.41 015.88-5.88a1 1 0 0 0 0-1.4117.5
4.9a1 1 0 0 0-1.4 0z"/><path d="M2 12V5a2 2 0 0 1 2-2h8"/><path d="M22 13v6a2 2 0 0 1-2 2H4a2 2 0 0 1-2-
2V11"/><circle cx="12" cy="12" r="1"/></svg>
294 |         <p>Enter roll numbers on the left and click Launch Session to begin.</p>
295 |     </div>
296 | </div>
297 | </div>
298 |
299 |     <div class="card card-attendance-undo-log glass-panel" data-glow>
300 |         <div class="card-header-accent">
301 |             <h3>Session Undo Stack (LIFO)</h3>
302 |             <p>Pop past decisions off the stack history to revert records.</p>
303 |         </div>
304 |         <div class="card-body">
305 |             <button id="btn-undo-attendance" class="btn btn-secondary" style="width:
100%; margin-bottom: 1.25rem; disabled>
306 |                 <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><path d="M21 17a9 9 0 0 0-9-9 9 9 0 0 0-6 2.3L3 13"/></svg>
307 |                 Undo Last Mark
308 |             </button>
309 |             <div id="undo-stack-preview">
310 |                 <div style="text-align: center; color: var(--text-muted); font-size:
0.85rem; font-style: italic; padding: 2rem 0; opacity: 0.5;">
311 |                     Stack is empty. No marks to undo.
312 |                 </div>
313 |             </div>
314 |         </div>
315 |     </div>
316 | </div>
317 | </section>
318 |
319 | <!-- Section: Sorting -->
320 | <section id="sec-sorting" class="content-section">
321 |     <div class="sorting-dashboard">
322 |         <div class="card card-sorting-controls glass-panel" data-glow>
323 |             <div class="card-header-accent">
324 |                 <h3>Visualizer Controls</h3>
325 |                 <p>Choose sorting criteria and trigger algorithm execution.</p>
326 |             </div>
327 |             <div class="card-body">
328 |                 <div class="form-group" style="margin-bottom: 1.5rem;">
329 |                     <label for="sort-criteria-select">Sort Criteria</label>
330 |                     <select id="sort-criteria-select">
331 |                         <option value="rollNo">Roll Number</option>
332 |                         <option value="name">Full Name</option>
333 |                         <option value="attendancePct">Attendance Percentage</option>
334 |                     </select>
335 |                 </div>
336 |                 <div style="display: flex; flex-direction: column; gap: 0.75rem;">
337 |                     <button id="btn-trigger-bubble" class="btn btn-secondary btn-sort-
trigger">
338 |                         <div class="sort-btn-content">
339 |                             <span>Bubble Sort</span>
340 |                             <span class="complexity-badge">O(N2)</span>
341 |                         </div>
342 |                     </button>
343 |                     <button id="btn-trigger-merge" class="btn btn-primary btn-sort-trigger">
344 |                         <div class="sort-btn-content">
345 |                             <span>Merge Sort</span>
346 |                             <span class="complexity-badge">O(N log N)</span>
347 |                         </div>
348 |                     </button>
349 |                 </div>
350 |                 <div style="margin-top: 1.5rem;">
351 |                     <label for="sort-speed-range" style="font-size: 0.85rem; color: var(--
text-muted); display: block; margin-bottom: 0.5rem; text-transform: uppercase; font-weight: 600;">Animation
Delay</label>
352 |                     <input type="range" id="sort-speed-range" min="100" max="1500"
step="100" value="400" style="width: 100%; accent-color: var(--primary);">

```

```

353 |                 <span id="speed-label" style="font-size: 0.75rem; color: var(--text-
muted); display: block; text-align: right; margin-top: 0.25rem;">400ms delay</span>
354 |                 </div>
355 |                 <div id="visualizer-status-board" style="display: none; margin-top: 1.5rem;
padding: 1rem; background: rgba(0,0,0,0.3); border-radius: 12px;">
356 |                     <div style="display: flex; justify-content: space-between; align-items:
center; margin-bottom: 0.5rem;">
357 |                         <strong style="font-size: 0.85rem; color: white; text-transform:
uppercase;">Status</strong>
358 |                         <span id="sort-timer-display" style="font-size: 0.75rem; color:
var(--accent); font-weight: 600;">0.00 ms</span>
359 |                     </div>
360 |                     <span id="sort-status-text" style="color: var(--primary-light); font-
weight: 700; font-size: 0.9rem;">Ready</span>
361 |                     <p id="sort-step-info" style="font-size: 0.8rem; color: var(--text-
muted); margin-top: 0.5rem; line-height: 1.4;"></p>
362 |                     </div>
363 |                 </div>
364 |             </div>
365 |
366 |             <div class="card glass-panel" style="flex: 1;" data-glow>
367 |                 <div class="card-header-accent">
368 |                     <h3>Database Array</h3>
369 |                     <p>Visual representation of students. Colors denote comparison and swap
actions.</p>
370 |                 </div>
371 |                 <div class="card-body">
372 |                     <div class="visualizer-array-wrapper" id="visualizer-array-wrapper">
373 |                         <!-- Rendered dynamically -->
374 |                     </div>
375 |                 </div>
376 |             </div>
377 |         </div>
378 |     </section>
379 |
380 |     <!-- Section: Reports -->
381 |     <section id="sec-reports" class="content-section">
382 |         <div class="stats-grid" style="margin-bottom: 2.5rem;">
383 |             <div class="stat-card glass-panel" data-glow>
384 |                 <div class="stat-icon" style="background: rgba(139, 92, 246, 0.1); border-color:
rgba(139, 92, 246, 0.3);">
385 |                     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
primary)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M17 21v-2a4 4 0 0 0 0-4-4H5a4 4 0
0 0 4-4v2"/><circle cx="9" cy="7" r="4"/></svg>
386 |                 </div>
387 |                 <div class="stat-details">
388 |                     <span class="stat-label">Total Enrollment</span>
389 |                     <h4 class="stat-value" id="rep-stat-total">0</h4>
390 |                 </div>
391 |             </div>
392 |             <div class="stat-card glass-panel" data-glow>
393 |                 <div class="stat-icon" style="background: rgba(16, 185, 129, 0.1); border-color:
rgba(16, 185, 129, 0.3);">
394 |                     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
success)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><polyline points="12 6 12 12 16 14"/></svg>
395 |                 </div>
396 |                 <div class="stat-details">
397 |                     <span class="stat-label">Average Rate</span>
398 |                     <h4 class="stat-value" id="rep-stat-avg">0.0%</h4>
399 |                 </div>
400 |             </div>
401 |             <div class="stat-card glass-panel" data-glow>
402 |                 <div class="stat-icon" style="background: rgba(239, 68, 68, 0.1); border-color:
rgba(239, 68, 68, 0.3);">
403 |                     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="var(--
danger)" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polygon points="7.86 2 16.14 2 22 7.86
22 16.14 16.14 22 7.86 22 2 16.14 2 7.86 7.86 2"/><line x1="12" y1="8" x2="12" y2="12"/><line x1="12" y1="16"
x2="12.01" y2="16"/></svg>
404 |                 </div>
405 |                 <div class="stat-details">
406 |                     <span class="stat-label">Deficient Students</span>
407 |                     <h4 class="stat-value" id="rep-stat-low">0</h4>

```

```

408 |         </div>
409 |     </div>
410 | </div>
411 |
412 |     <div class="reports-container-grid">
413 |         <div class="card glass-panel" data-glow>
414 |             <div class="card-header-accent">
415 |                 <h3>Attendance Analytics</h3>
416 |                 <p>Visualizing individual class presence rates mapped to the 75% target
threshold.</p>
417 |             </div>
418 |             <div class="card-body" style="height: 350px; position: relative;">
419 |                 <canvas id="attendance-bar-chart"></canvas>
420 |             </div>
421 |         </div>
422 |
423 |         <div class="card glass-panel" style="margin-top: 2rem;" data-glow>
424 |             <div class="card-header-accent" style="display: flex; justify-content: space-
between; align-items: center; flex-wrap: wrap; gap: 1rem;">
425 |                 <div>
426 |                     <h3>Class Records</h3>
427 |                     <p>Toggle filters to check general directories vs deficit listings.</p>
428 |                 </div>
429 |                 <div style="display: flex; gap: 0.75rem; align-items: center;">
430 |                     <button id="btn-export-csv-rep" class="btn btn-secondary"
style="padding: 0.5rem 1.25rem; font-size: 0.85rem; display: flex; align-items: center; gap: 0.5rem;">
431 |                         <svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><path d="M21 15v4a2 2 0 0
1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
432 |                         CSV
433 |                     </button>
434 |                     <div class="tab-button-group">
435 |                         <button class="tab-btn active" id="btn-tab-all-records">All
Students</button>
436 |                         <button class="tab-btn" id="btn-tab-deficient-records">Low
Attendance</button>
437 |                     </div>
438 |                 </div>
439 |             </div>
440 |             <div class="card-body scroll-x">
441 |                 <table class="table">
442 |                     <thead>
443 |                         <tr id="table-reports-header"></tr>
444 |                     </thead>
445 |                     <tbody id="reports-tbody"></tbody>
446 |                 </table>
447 |             </div>
448 |         </div>
449 |     </div>
450 | </section>
451 |
452 | <!-- Section: About -->
453 | <section id="sec-about" class="content-section">
454 |     <div class="card glass-panel" style="margin-bottom: 2.5rem;" data-glow>
455 |         <div class="card-header-accent">
456 |             <h3>Computer Science Abstractions</h3>
457 |             <p>Overview of the data structures and algorithms implemented.</p>
458 |         </div>
459 |         <div class="card-body">
460 |             <div class="dsa-grid">
461 |                 <div class="dsa-card glass-panel" data-glow>
462 |                     <div class="dsa-card-icon" style="color: var(--primary-light);">
463 |                         <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><rect x="3" y="3" width="18"
height="18" rx="2" ry="2"/><line x1="9" y1="3" x2="9" y2="21"/><line x1="15" y1="3" x2="15" y2="21"/></svg>
464 |                     </div>
465 |                     <div class="dsa-card-content">
466 |                         <h4>Contiguous Store (std::vector)</h4>
467 |                         <p><strong>C++</strong> <code>std::vector<Student></code></p>
468 |                         <small>Maintains Contiguous memory blocks for student lists,
enabling O(1) index access.</small>
469 |                     </div>
470 |                 </div>

```

```

471 |
472 |         <div class="dsa-card glass-panel" data-glow>
473 |             <div class="dsa-card-icon" style="color: var(--accent);">
474 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><path d="M8 12h8"/><path d="M12 8v8"/></svg>
475 |             </div>
476 |             <div class="dsa-card-content">
477 |                 <h4>Hash Index Map (unordered_map)</h4>
478 |                 <p><strong>C++:</strong> <code>std::unordered_map</code></p>
479 |                 <small>Maps roll numbers to array indexes for O(1) average time
lookups.</small>
480 |             </div>
481 |         </div>
482 |
483 |         <div class="dsa-card glass-panel" data-glow>
484 |             <div class="dsa-card-icon" style="color: var(--success);">
485 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 11.08V12a10 10
0 1 1-5.93-9.14"/><polyline points="22 4 12 14.01 9 11.01"/></svg>
486 |             </div>
487 |             <div class="dsa-card-content">
488 |                 <h4>FIFO Session Queue (std::queue)</h4>
489 |                 <p><strong>C++:</strong> <code>std::queue<int></code></p>
490 |                 <small>Processes batches of students in First-In-First-Out order
during marking.</small>
491 |             </div>
492 |         </div>
493 |
494 |         <div class="dsa-card glass-panel" data-glow>
495 |             <div class="dsa-card-icon" style="color: var(--danger);">
496 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21.5 2v6h-6M21.34
15.57a10 10 0 1 1-.57-8.38l5.67-5.67"/></svg>
497 |             </div>
498 |             <div class="dsa-card-content">
499 |                 <h4>LIFO Undo Stack (std::stack)</h4>
500 |                 <p><strong>C++:</strong> <code>std::stack</code></p>
501 |                 <small>Records attendance marks chronologically enabling LIFO
undo.</small>
502 |             </div>
503 |         </div>
504 |
505 |         <div class="dsa-card glass-panel" data-glow>
506 |             <div class="dsa-card-icon" style="color: var(--warning);">
507 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
508 |             </div>
509 |             <div class="dsa-card-content">
510 |                 <h4>Binary Search (O(log N))</h4>
511 |                 <p><strong>Prerequisite:</strong> Sorted dataset</p>
512 |                 <small>Halves the search space at each comparison step. Runs in
logarithmic time.</small>
513 |             </div>
514 |         </div>
515 |
516 |         <div class="dsa-card glass-panel" data-glow>
517 |             <div class="dsa-card-icon" style="color: #60a5fa;">
518 |                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><line x1="18" y1="20"
x2="12" y2="4"/><line x1="6" y1="20" x2="6" y2="14"/></svg>
519 |             </div>
520 |             <div class="dsa-card-content">
521 |                 <h4>Sorting Visualizer</h4>
522 |                 <p><strong>Bubble:</strong> O(N2) <br><strong>Merge:</strong> O(N
log N)</p>
523 |                 <small>Compares quadratic scaling against logarithmic recursively
split sorting.</small>
524 |             </div>
525 |         </div>
526 |     </div>
527 | </div>

```

```
528 |         </div>
529 |     </section>
530 | </div>
531 | </main>
532 | </div>
533 |
534 |     <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
535 |     <script src="app.js"></script>
536 | </body>
537 | </html>
```

style.css

CSS — 1090 lines

```

1 | @import
2 | url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&display=swap');
3 |
4 | :root {
5 |     /* Color Palette - Modern Dark/Neon Aesthetic */
6 |     --bg-dark: #070913;
7 |     --bg-panel: rgba(17, 20, 36, 0.65);
8 |     --bg-panel-hover: rgba(28, 32, 54, 0.75);
9 |     --border-glass: rgba(255, 255, 255, 0.06);
10 |     --border-glass-glow: rgba(139, 92, 246, 0.3);
11 |
12 |     --primary: #8b5cf6; /* Vibrant Purple */
13 |     --primary-hover: #7c3aed;
14 |     --primary-light: #a78bfa;
15 |     --primary-glow: rgba(139, 92, 246, 0.4);
16 |
17 |     --accent: #0ea5e9; /* Cyan */
18 |     --accent-glow: rgba(14, 165, 233, 0.4);
19 |
20 |     --success: #10b981; /* Emerald */
21 |     --success-glow: rgba(16, 185, 129, 0.4);
22 |
23 |     --danger: #ef4444; /* Rose */
24 |     --danger-glow: rgba(239, 68, 68, 0.4);
25 |
26 |     --warning: #f59e0b; /* Amber */
27 |
28 |     --text-main: #f8fafc;
29 |     --text-muted: #64748b;
30 |
31 |     /* Typography */
32 |     --font-heading: 'Outfit', sans-serif;
33 |     --font-body: 'Plus Jakarta Sans', sans-serif;
34 |
35 |     /* Shadows & Transitions */
36 |     --shadow-glass: 0 8px 32px 0 rgba(0, 0, 0, 0.45);
37 |     --shadow-neon: 0 0 20px var(--primary-glow);
38 |     --transition-fast: 0.15s cubic-bezier(0.4, 0, 0.2, 1);
39 |     --transition-smooth: 0.3s cubic-bezier(0.4, 0, 0.2, 1);
40 |     --transition-bounce: 0.4s cubic-bezier(0.34, 1.56, 0.64, 1);
41 | }
42 |
43 | * {
44 |     margin: 0;
45 |     padding: 0;
46 |     box-sizing: border-box;
47 | }
48 |
49 | body {
50 |     font-family: var(--font-body);
51 |     background-color: var(--bg-dark);
52 |     color: var(--text-main);
53 |     line-height: 1.6;
54 |     overflow-x: hidden;
55 |     min-height: 100vh;
56 | }
57 |
58 | /* --- Particle Canvas --- */
59 | #particle-canvas {
60 |     position: fixed;
61 |     top: 0;
62 |     left: 0;
63 |     width: 100vw;
64 |     height: 100vh;
65 |     z-index: -2;
66 |     pointer-events: none;

```

```

66 |     opacity: 0.3;
67 | }
68 |
69 | /* --- Animated Ambient Blur Blobs --- */
70 | body::before, body::after {
71 |     content: '';
72 |     position: fixed;
73 |     border-radius: 50%;
74 |     filter: blur(140px);
75 |     z-index: -3;
76 |     animation: float 25s infinite alternate ease-in-out;
77 | }
78 |
79 | body::before {
80 |     width: 700px;
81 |     height: 700px;
82 |     background: radial-gradient(circle, var(--primary-glow), transparent 75%);
83 |     top: -250px;
84 |     left: -150px;
85 | }
86 |
87 | body::after {
88 |     width: 600px;
89 |     height: 600px;
90 |     background: radial-gradient(circle, var(--accent-glow), transparent 75%);
91 |     bottom: -200px;
92 |     right: -150px;
93 |     animation-delay: -12s;
94 | }
95 |
96 | @keyframes float {
97 |     0% { transform: translate(0, 0) scale(1); }
98 |     50% { transform: translate(80px, 40px) scale(1.08); }
99 |     100% { transform: translate(-40px, 120px) scale(0.92); }
100 | }
101 |
102 | /* --- App Layout --- */
103 | .app-container {
104 |     display: flex;
105 |     min-height: 100vh;
106 |     padding: 1.5rem;
107 |     gap: 1.5rem;
108 |     max-width: 1800px;
109 |     margin: 0 auto;
110 | }
111 |
112 | /* --- Glassmorphism Utilities --- */
113 | .glass-panel {
114 |     background: var(--bg-panel);
115 |     backdrop-filter: blur(20px);
116 |     -webkit-backdrop-filter: blur(20px);
117 |     border: 1px solid var(--border-glass);
118 |     border-radius: 24px;
119 |     box-shadow: var(--shadow-glass);
120 |     transition: border-color var(--transition-smooth), box-shadow var(--transition-smooth);
121 | }
122 |
123 | /* --- Hover Spotlight Glow Effect --- */
124 | [data-glow] {
125 |     position: relative;
126 | }
127 | [data-glow]::before {
128 |     content: '';
129 |     position: absolute;
130 |     top: 0; left: 0; right: 0; bottom: 0;
131 |     border-radius: inherit;
132 |     background: radial-gradient(600px circle at var(--mouse-x, 0) var(--mouse-y, 0), rgba(139, 92, 246,
0.08), transparent 40%);
133 |     z-index: 1;
134 |     pointer-events: none;
135 |     transition: opacity 0.4s ease;
136 |     opacity: 0;
137 |     border: 1px solid transparent;

```

```

138 | }
139 | [data-glow]:hover::before {
140 |     opacity: 1;
141 | }
142 | [data-glow] > * {
143 |     position: relative;
144 |     z-index: 2;
145 | }
146 |
147 | /* --- Sidebar Navigation --- */
148 | .sidebar {
149 |     width: 280px;
150 |     display: flex;
151 |     flex-direction: column;
152 |     padding: 2rem 1.5rem;
153 |     flex-shrink: 0;
154 |     transition: var(--transition-smooth);
155 |     position: relative;
156 |     overflow: hidden;
157 | }
158 |
159 | .sidebar-header {
160 |     display: flex;
161 |     align-items: center;
162 |     gap: 1rem;
163 |     margin-bottom: 3rem;
164 | }
165 |
166 | .logo-icon {
167 |     width: 45px;
168 |     height: 45px;
169 |     background: linear-gradient(135deg, var(--primary), var(--accent));
170 |     border-radius: 14px;
171 |     display: flex;
172 |     align-items: center;
173 |     justify-content: center;
174 |     box-shadow: var(--shadow-neon);
175 |     color: white;
176 | }
177 |
178 | .sidebar-header h2 {
179 |     font-family: var(--font-heading);
180 |     font-size: 1.5rem;
181 |     font-weight: 800;
182 |     letter-spacing: -0.5px;
183 |     background: linear-gradient(to right, #fff, #94a3b8);
184 |     -webkit-background-clip: text;
185 |     -webkit-text-fill-color: transparent;
186 | }
187 |
188 | .sidebar-menu {
189 |     display: flex;
190 |     flex-direction: column;
191 |     gap: 0.5rem;
192 |     flex: 1;
193 | }
194 |
195 | .menu-item {
196 |     display: flex;
197 |     align-items: center;
198 |     gap: 1rem;
199 |     padding: 1rem 1.25rem;
200 |     border: none;
201 |     background: transparent;
202 |     color: var(--text-muted);
203 |     font-family: var(--font-body);
204 |     font-size: 1rem;
205 |     font-weight: 500;
206 |     border-radius: 12px;
207 |     cursor: pointer;
208 |     transition: var(--transition-bounce);
209 |     text-align: left;
210 |     position: relative;

```

```

211 |     overflow: hidden;
212 | }
213 |
214 | .menu-item:hover {
215 |     color: var(--text-main);
216 |     background: rgba(255, 255, 255, 0.02);
217 |     transform: translateX(6px);
218 | }
219 |
220 | .menu-item.active {
221 |     color: white;
222 |     background: linear-gradient(90deg, rgba(139, 92, 246, 0.12), transparent);
223 |     border-left: 4px solid var(--primary);
224 | }
225 |
226 | .menu-item svg {
227 |     transition: var(--transition-bounce);
228 | }
229 |
230 | .menu-item.active svg {
231 |     color: var(--primary-light);
232 |     transform: scale(1.1);
233 |     filter: drop-shadow(0 0 5px var(--primary-glow));
234 | }
235 |
236 | .sidebar-footer {
237 |     margin-top: auto;
238 |     padding-top: 2rem;
239 |     border-top: 1px solid var(--border-glass);
240 |     text-align: center;
241 | }
242 |
243 | .sidebar-footer p {
244 |     font-size: 0.8rem;
245 |     color: var(--text-muted);
246 |     font-weight: 600;
247 |     text-transform: uppercase;
248 |     letter-spacing: 1px;
249 | }
250 |
251 | .version {
252 |     display: inline-block;
253 |     margin-top: 0.5rem;
254 |     padding: 0.2rem 0.6rem;
255 |     background: rgba(139, 92, 246, 0.1);
256 |     color: var(--primary-light);
257 |     border-radius: 20px;
258 |     font-size: 0.7rem;
259 |     font-weight: 700;
260 |     border: 1px solid rgba(139, 92, 246, 0.15);
261 | }
262 |
263 | /* --- Main Content Area --- */
264 | .main-content {
265 |     flex: 1;
266 |     display: flex;
267 |     flex-direction: column;
268 |     min-width: 0;
269 | }
270 |
271 | /* --- Header --- */
272 | .content-header {
273 |     display: flex;
274 |     justify-content: space-between;
275 |     align-items: center;
276 |     margin-bottom: 2.5rem;
277 |     padding: 1.5rem 2.5rem;
278 | }
279 |
280 | .header-title h1 {
281 |     font-family: var(--font-heading);
282 |     font-size: 2.5rem;
283 |     font-weight: 800;

```

```

284 |     margin-bottom: 0.25rem;
285 |     letter-spacing: -0.5px;
286 | }
287 |
288 | .header-title p {
289 |     color: var(--text-muted);
290 |     font-size: 0.95rem;
291 | }
292 |
293 | .theme-pill {
294 |     background: rgba(16, 185, 129, 0.08);
295 |     border: 1px solid rgba(16, 185, 129, 0.15);
296 |     padding: 0.5rem 1rem;
297 |     border-radius: 30px;
298 |     display: flex;
299 |     align-items: center;
300 |     gap: 0.5rem;
301 | }
302 |
303 | .pill-badge {
304 |     color: var(--success);
305 |     font-weight: 600;
306 |     font-size: 0.85rem;
307 |     display: flex;
308 |     align-items: center;
309 | }
310 |
311 | .pill-badge::before {
312 |     content: '';
313 |     display: inline-block;
314 |     width: 8px;
315 |     height: 8px;
316 |     background-color: var(--success);
317 |     border-radius: 50%;
318 |     margin-right: 8px;
319 |     box-shadow: 0 0 10px var(--success);
320 |     animation: pulse 2s infinite;
321 | }
322 |
323 | /* --- Content Sections Transitions --- */
324 | .content-section {
325 |     display: none;
326 |     opacity: 0;
327 |     transform: translateY(12px) scale(0.99);
328 |     transition: opacity 0.35s cubic-bezier(0.4, 0, 0.2, 1), transform 0.35s cubic-bezier(0.4, 0, 0.2, 1);
329 | }
330 |
331 | .content-section.active {
332 |     display: block;
333 |     opacity: 1;
334 |     transform: translateY(0) scale(1);
335 | }
336 |
337 | /* --- Dashboard Section --- */
338 | .welcome-card {
339 |     display: flex;
340 |     justify-content: space-between;
341 |     align-items: center;
342 |     padding: 3rem;
343 |     margin-bottom: 2.5rem;
344 |     position: relative;
345 |     overflow: hidden;
346 | }
347 |
348 | .welcome-card::after {
349 |     content: '';
350 |     position: absolute;
351 |     top: 0; right: 0; bottom: 0; left: 0;
352 |     background: linear-gradient(135deg, rgba(139, 92, 246, 0.12), transparent);
353 |     pointer-events: none;
354 | }
355 |
356 | .welcome-text h3 {

```

```

357 |     font-family: var(--font-heading);
358 |     font-size: 2.2rem;
359 |     margin-bottom: 1rem;
360 |     font-weight: 800;
361 | }
362 |
363 | .welcome-text p {
364 |     color: var(--text-muted);
365 |     font-size: 1.1rem;
366 |     max-width: 600px;
367 | }
368 |
369 | .circle-glow {
370 |     width: 150px;
371 |     height: 150px;
372 |     background: radial-gradient(circle, var(--primary), transparent 70%);
373 |     border-radius: 50%;
374 |     filter: blur(35px);
375 |     animation: pulseGlow 4s infinite alternate ease-in-out;
376 | }
377 |
378 | .stats-grid {
379 |     display: grid;
380 |     grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
381 |     gap: 1.5rem;
382 | }
383 |
384 | .stat-card {
385 |     display: flex;
386 |     align-items: center;
387 |     gap: 1.5rem;
388 |     padding: 1.75rem;
389 |     transition: var(--transition-bounce), border-color var(--transition-smooth);
390 |     background: var(--bg-panel);
391 |     border: 1px solid var(--border-glass);
392 |     border-radius: 20px;
393 | }
394 |
395 | .stat-card:hover {
396 |     transform: translateY(-6px);
397 |     border-color: rgba(139, 92, 246, 0.25);
398 |     box-shadow: 0 12px 30px rgba(0,0,0,0.3);
399 | }
400 |
401 | .stat-icon {
402 |     width: 60px;
403 |     height: 60px;
404 |     border-radius: 16px;
405 |     display: flex;
406 |     align-items: center;
407 |     justify-content: center;
408 |     background: rgba(255, 255, 255, 0.02);
409 |     border: 1px solid rgba(255,255,255,0.04);
410 |     transition: var(--transition-fast);
411 | }
412 |
413 | .stat-card:hover .stat-icon {
414 |     background: rgba(139, 92, 246, 0.1);
415 |     border-color: rgba(139, 92, 246, 0.2);
416 | }
417 |
418 | .stat-details .stat-label {
419 |     display: block;
420 |     color: var(--text-muted);
421 |     font-size: 0.85rem;
422 |     font-weight: 700;
423 |     text-transform: uppercase;
424 |     letter-spacing: 0.5px;
425 |     margin-bottom: 0.25rem;
426 | }
427 |
428 | .stat-details .stat-value {
429 |     font-family: var(--font-heading);

```

```

430 |     font-size: 2.2rem;
431 |     font-weight: 800;
432 |     background: linear-gradient(to right, #fff, #94a3b8);
433 |     -webkit-background-clip: text;
434 |     -webkit-text-fill-color: transparent;
435 | }
436 |
437 | /* --- Split Layouts --- */
438 | .form-table-split, .search-container-grid, .reports-container-grid, .attendance-dashboard, .sorting-
dashboard {
439 |     display: grid;
440 |     gap: 1.5rem;
441 | }
442 |
443 | .form-table-split { grid-template-columns: 1fr 2.2fr; }
444 | .search-container-grid { grid-template-columns: 1fr 2fr; }
445 | .attendance-dashboard { grid-template-columns: 1fr 1fr 1fr; }
446 | .sorting-dashboard { grid-template-columns: 350px 1fr; }
447 |
448 | @media (max-width: 1200px) {
449 |     .form-table-split, .search-container-grid, .sorting-dashboard { grid-template-columns: 1fr; }
450 |     .attendance-dashboard { grid-template-columns: 1fr; }
451 | }
452 |
453 | /* --- Cards --- */
454 | .card {
455 |     background: var(--bg-panel);
456 |     border: 1px solid var(--border-glass);
457 |     border-radius: 20px;
458 |     overflow: hidden;
459 |     display: flex;
460 |     flex-direction: column;
461 | }
462 |
463 | .card-header-accent {
464 |     padding: 1.5rem;
465 |     border-bottom: 1px solid var(--border-glass);
466 |     background: linear-gradient(180deg, rgba(255,255,255,0.02), transparent);
467 | }
468 |
469 | .card-header-accent h3 {
470 |     font-family: var(--font-heading);
471 |     font-size: 1.25rem;
472 |     font-weight: 700;
473 |     margin-bottom: 0.25rem;
474 |     color: #fff;
475 | }
476 |
477 | .card-header-accent p {
478 |     font-size: 0.85rem;
479 |     color: var(--text-muted);
480 | }
481 |
482 | .card-body {
483 |     padding: 1.5rem;
484 |     flex: 1;
485 | }
486 |
487 | /* --- Forms & Inputs --- */
488 | .student-form {
489 |     display: flex;
490 |     flex-direction: column;
491 |     gap: 1.25rem;
492 | }
493 |
494 | .form-group-row {
495 |     display: grid;
496 |     grid-template-columns: 1fr 1fr;
497 |     gap: 1rem;
498 | }
499 |
500 | .form-group label {
501 |     display: block;

```

```

502 |     font-size: 0.8rem;
503 |     font-weight: 700;
504 |     color: var(--text-muted);
505 |     margin-bottom: 0.5rem;
506 |     text-transform: uppercase;
507 |     letter-spacing: 0.5px;
508 | }
509 |
510 | input[type="text"],
511 | input[type="number"],
512 | select,
513 | textarea {
514 |     width: 100%;
515 |     background: rgba(0, 0, 0, 0.25);
516 |     border: 1px solid var(--border-glass);
517 |     color: white;
518 |     padding: 0.85rem 1rem;
519 |     border-radius: 12px;
520 |     font-family: var(--font-body);
521 |     font-size: 0.95rem;
522 |     transition: var(--transition-fast);
523 | }
524 |
525 | input:focus, select:focus, textarea:focus {
526 |     outline: none;
527 |     border-color: var(--primary-light);
528 |     background: rgba(0, 0, 0, 0.4);
529 |     box-shadow: 0 0 0 3px rgba(139, 92, 246, 0.15);
530 | }
531 |
532 | input::placeholder, textarea::placeholder {
533 |     color: rgba(148, 163, 184, 0.35);
534 | }
535 |
536 | /* --- Buttons --- */
537 | .btn {
538 |     padding: 0.85rem 1.5rem;
539 |     border-radius: 12px;
540 |     font-family: var(--font-heading);
541 |     font-weight: 700;
542 |     font-size: 0.95rem;
543 |     cursor: pointer;
544 |     display: inline-flex;
545 |     align-items: center;
546 |     justify-content: center;
547 |     gap: 0.5rem;
548 |     transition: var(--transition-bounce);
549 |     border: none;
550 | }
551 |
552 | .btn:active {
553 |     transform: scale(0.95);
554 | }
555 |
556 | .btn-primary {
557 |     background: linear-gradient(135deg, var(--primary), var(--primary-hover));
558 |     color: white;
559 |     box-shadow: 0 4px 15px rgba(139, 92, 246, 0.2);
560 | }
561 |
562 | .btn-primary:hover {
563 |     box-shadow: 0 6px 22px rgba(139, 92, 246, 0.4);
564 |     transform: translateY(-2px);
565 | }
566 |
567 | .btn-secondary {
568 |     background: rgba(255, 255, 255, 0.04);
569 |     color: white;
570 |     border: 1px solid rgba(255, 255, 255, 0.08);
571 | }
572 |
573 | .btn-secondary:hover {
574 |     background: rgba(255, 255, 255, 0.08);

```

```

575 |     border-color: rgba(255, 255, 255, 0.15);
576 |     transform: translateY(-1px);
577 | }
578 |
579 | .form-actions {
580 |     display: flex;
581 |     gap: 1rem;
582 |     margin-top: 1rem;
583 | }
584 |
585 | .form-actions .btn {
586 |     flex: 1;
587 | }
588 |
589 | /* --- Tables --- */
590 | .scroll-x {
591 |     overflow-x: auto;
592 | }
593 |
594 | .table {
595 |     width: 100%;
596 |     border-collapse: separate;
597 |     border-spacing: 0 0.5rem;
598 | }
599 |
600 | .table th {
601 |     text-align: left;
602 |     padding: 1rem;
603 |     color: var(--text-muted);
604 |     font-weight: 700;
605 |     font-size: 0.8rem;
606 |     text-transform: uppercase;
607 |     letter-spacing: 0.5px;
608 |     border-bottom: 1px solid var(--border-glass);
609 | }
610 |
611 | .table td {
612 |     padding: 1rem;
613 |     background: rgba(255, 255, 255, 0.015);
614 |     transition: var(--transition-fast);
615 | }
616 |
617 | .table tbody tr {
618 |     transition: var(--transition-bounce);
619 | }
620 |
621 | .table tbody tr:hover td {
622 |     background: rgba(255, 255, 255, 0.04);
623 |     transform: translateY(-1px);
624 | }
625 |
626 | .table td:first-child { border-top-left-radius: 12px; border-bottom-left-radius: 12px; }
627 | .table td:last-child { border-top-right-radius: 12px; border-bottom-right-radius: 12px; }
628 |
629 | .text-right { text-align: right; }
630 |
631 | /* Badges */
632 | .badge {
633 |     padding: 0.35rem 0.75rem;
634 |     border-radius: 20px;
635 |     font-size: 0.75rem;
636 |     font-weight: 700;
637 |     letter-spacing: 0.5px;
638 |     display: inline-block;
639 | }
640 |
641 | .badge-success {
642 |     background: rgba(16, 185, 129, 0.08);
643 |     color: var(--success);
644 |     border: 1px solid rgba(16, 185, 129, 0.15);
645 | }
646 |
647 | .badge-danger {

```

```

648 |     background: rgba(239, 68, 68, 0.08);
649 |     color: var(--danger);
650 |     border: 1px solid rgba(239, 68, 68, 0.15);
651 | }
652 |
653 | /* Table Actions */
654 | .btn-action-group {
655 |     display: flex;
656 |     gap: 0.5rem;
657 |     justify-content: flex-end;
658 | }
659 |
660 | .btn-action {
661 |     width: 32px;
662 |     height: 32px;
663 |     border-radius: 8px;
664 |     border: none;
665 |     display: flex;
666 |     align-items: center;
667 |     justify-content: center;
668 |     cursor: pointer;
669 |     background: rgba(255, 255, 255, 0.04);
670 |     color: var(--text-muted);
671 |     transition: var(--transition-bounce);
672 | }
673 |
674 | .btn-action.btn-edit:hover { background: rgba(14, 165, 233, 0.15); color: var(--accent); }
675 | .btn-action.btn-delete:hover { background: rgba(239, 68, 68, 0.15); color: var(--danger); }
676 |
677 | /* --- Banners --- */
678 | .error-banner, .success-banner, .search-warning-banner {
679 |     padding: 1rem;
680 |     border-radius: 12px;
681 |     font-size: 0.9rem;
682 |     font-weight: 600;
683 |     display: flex;
684 |     align-items: center;
685 |     gap: 0.75rem;
686 |     animation: slideDown 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
687 | }
688 |
689 | .error-banner {
690 |     background: rgba(239, 68, 68, 0.08);
691 |     border: 1px solid rgba(239, 68, 68, 0.15);
692 |     color: #fca5a5;
693 | }
694 |
695 | .success-banner {
696 |     background: rgba(16, 185, 129, 0.08);
697 |     border: 1px solid rgba(16, 185, 129, 0.15);
698 |     color: #6ee7b7;
699 | }
700 |
701 | .search-warning-banner {
702 |     background: rgba(245, 158, 11, 0.08);
703 |     border: 1px solid rgba(245, 158, 11, 0.15);
704 |     color: #fcd34d;
705 | }
706 |
707 | /* --- Radio Pills (Search Form) --- */
708 | .radio-pill-group {
709 |     display: flex;
710 |     gap: 0.5rem;
711 | }
712 |
713 | .radio-pill {
714 |     flex: 1;
715 |     position: relative;
716 |     cursor: pointer;
717 | }
718 |
719 | .radio-pill input {
720 |     position: absolute;

```

```

721 |     opacity: 0;
722 | }
723 |
724 | .radio-pill span {
725 |     display: block;
726 |     text-align: center;
727 |     padding: 0.85rem;
728 |     background: rgba(255, 255, 255, 0.03);
729 |     border: 1px solid var(--border-glass);
730 |     border-radius: 12px;
731 |     font-size: 0.9rem;
732 |     font-weight: 700;
733 |     color: var(--text-muted);
734 |     transition: var(--transition-bounce);
735 | }
736 |
737 | .radio-pill input:checked + span {
738 |     background: rgba(139, 92, 246, 0.12);
739 |     border-color: var(--primary-light);
740 |     color: white;
741 |     box-shadow: 0 0 15px rgba(139, 92, 246, 0.15);
742 | }
743 |
744 | /* --- Benchmarks & Stopwatch timers --- */
745 | .timer-badge {
746 |     background: rgba(14, 165, 233, 0.08);
747 |     border: 1px solid rgba(14, 165, 233, 0.15);
748 |     padding: 0.35rem 0.75rem;
749 |     border-radius: 20px;
750 |     color: var(--accent);
751 |     font-size: 0.8rem;
752 |     font-weight: 700;
753 |     display: inline-flex;
754 |     align-items: center;
755 |     gap: 0.4rem;
756 |     animation: pulseTimer 2s infinite;
757 | }
758 |
759 | @keyframes pulseTimer {
760 |     0% { box-shadow: 0 0 0 0 rgba(14, 165, 233, 0.3); }
761 |     70% { box-shadow: 0 0 0 8px rgba(14, 165, 233, 0); }
762 |     100% { box-shadow: 0 0 0 0 rgba(14, 165, 233, 0); }
763 | }
764 |
765 | .benchmark-board {
766 |     display: grid;
767 |     grid-template-columns: 1fr 1fr;
768 |     gap: 1rem;
769 |     margin-top: 1.5rem;
770 | }
771 |
772 | .benchmark-card {
773 |     background: rgba(0, 0, 0, 0.2);
774 |     border: 1px solid var(--border-glass);
775 |     border-radius: 12px;
776 |     padding: 1.25rem;
777 |     display: flex;
778 |     flex-direction: column;
779 |     align-items: center;
780 |     text-align: center;
781 | }
782 |
783 | .benchmark-card.metric-highlight {
784 |     background: linear-gradient(135deg, rgba(139, 92, 246, 0.08), rgba(14, 165, 233, 0.08));
785 |     border-color: rgba(139, 92, 246, 0.25);
786 | }
787 |
788 | .bench-title { font-size: 0.75rem; text-transform: uppercase; font-weight: 700; color: var(--text-muted);
margin-bottom: 0.5rem; }
789 | .bench-value { font-family: var(--font-heading); font-size: 2.2rem; font-weight: 800; color: white; line-
height: 1; margin-bottom: 0.5rem; }
790 | .bench-desc { font-size: 0.8rem; color: var(--text-muted); }
791 |

```

```

792 | .search-result-card {
793 |     display: flex;
794 |     flex-direction: column;
795 |     gap: 1.5rem;
796 |     animation: slideUp 0.35s ease-out;
797 | }
798 |
799 | .result-main-info {
800 |     display: flex;
801 |     justify-content: space-between;
802 |     align-items: center;
803 |     background: rgba(255, 255, 255, 0.02);
804 |     padding: 1.5rem;
805 |     border-radius: 16px;
806 |     border: 1px solid var(--border-glass);
807 | }
808 |
809 | .result-meta-name h4 { font-size: 1.5rem; margin-bottom: 0.25rem; font-family: var(--font-heading); font-
weight: 800; }
810 | .result-meta-name span { color: var(--text-muted); font-size: 0.9rem; }
811 | .result-pct-value { font-size: 2rem; font-weight: 800; font-family: var(--font-heading); }
812 |
813 | /* --- Sorting Visualizer --- */
814 | .visualizer-array-wrapper {
815 |     display: flex;
816 |     flex-direction: column;
817 |     gap: 0.75rem;
818 |     height: 100%;
819 |     min-height: 400px;
820 | }
821 |
822 | .visualizer-element-card {
823 |     display: flex;
824 |     align-items: center;
825 |     background: rgba(255, 255, 255, 0.015);
826 |     border: 1px solid rgba(255,255,255,0.03);
827 |     border-radius: 12px;
828 |     padding: 0.75rem 1rem;
829 |     gap: 1rem;
830 |     transition: all 0.35s cubic-bezier(0.25, 0.8, 0.25, 1);
831 |     position: relative;
832 |     overflow: hidden;
833 | }
834 |
835 | .element-index { width: 60px; font-size: 0.75rem; color: var(--text-muted); font-weight: 700; text-
transform: uppercase; }
836 | .element-meta { width: 180px; display: flex; flex-direction: column; }
837 | .element-meta strong { font-size: 0.95rem; }
838 | .element-meta span { font-size: 0.75rem; color: var(--text-muted); }
839 |
840 | .element-bar-container {
841 |     flex: 1;
842 |     height: 12px;
843 |     background: rgba(0,0,0,0.3);
844 |     border-radius: 6px;
845 |     overflow: hidden;
846 | }
847 |
848 | .element-bar {
849 |     height: 100%;
850 |     background: linear-gradient(90deg, var(--primary), var(--accent));
851 |     border-radius: 6px;
852 |     transition: width 0.35s cubic-bezier(0.25, 0.8, 0.25, 1), background 0.25s;
853 |     box-shadow: 0 0 10px var(--primary-glow);
854 | }
855 |
856 | .element-value { width: 60px; text-align: right; font-weight: 800; font-family: var(--font-heading); }
857 |
858 | /* Visualization States */
859 | .visualizer-element-card.comparing {
860 |     background: rgba(245, 158, 11, 0.08);
861 |     border-color: var(--warning);
862 |     transform: scale(1.01) translateX(4px);

```

```

863 |     z-index: 10;
864 | }
865 | .visualizer-element-card.comparing .element-bar { background: var(--warning); box-shadow: 0 0 15px
866 | rgba(245, 158, 11, 0.4); }
867 | .visualizer-element-card.swapping {
868 |     background: rgba(239, 68, 68, 0.08);
869 |     border-color: var(--danger);
870 |     transform: scale(1.01) translateX(-4px);
871 |     z-index: 10;
872 | }
873 | .visualizer-element-card.swapping .element-bar { background: var(--danger); box-shadow: 0 0 15px rgba(239,
874 | 68, 68, 0.4); }
875 | .visualizer-element-card.sorted-highlight {
876 |     background: rgba(16, 185, 129, 0.08);
877 |     border-color: var(--success);
878 | }
879 | .visualizer-element-card.sorted-highlight .element-bar { background: var(--success); box-shadow: 0 0 15px
880 | rgba(16, 185, 129, 0.4); }
881 | .sort-btn-content {
882 |     display: flex;
883 |     justify-content: space-between;
884 |     align-items: center;
885 |     width: 100%;
886 | }
887 |
888 | .complexity-badge {
889 |     font-size: 0.7rem;
890 |     padding: 0.2rem 0.5rem;
891 |     border-radius: 4px;
892 |     background: rgba(0,0,0,0.3);
893 | }
894 |
895 | /* --- Attendance Queue & Marking --- */
896 | .queue-pill {
897 |     background: rgba(255,255,255,0.04);
898 |     border: 1px solid var(--border-glass);
899 |     padding: 0.4rem 0.8rem;
900 |     border-radius: 20px;
901 |     font-size: 0.85rem;
902 |     display: flex;
903 |     align-items: center;
904 |     gap: 0.4rem;
905 |     transition: var(--transition-bounce);
906 | }
907 |
908 | .queue-pill.current {
909 |     background: rgba(14, 165, 233, 0.12);
910 |     border-color: var(--accent);
911 |     color: white;
912 |     box-shadow: 0 0 10px rgba(14, 165, 233, 0.2);
913 |     transform: scale(1.05);
914 | }
915 |
916 | .active-marking-card {
917 |     width: 100%;
918 |     max-width: 400px;
919 |     background: rgba(0,0,0,0.25);
920 |     border: 1px solid var(--border-glass);
921 |     border-radius: 24px;
922 |     padding: 2.5rem 2rem;
923 |     display: flex;
924 |     flex-direction: column;
925 |     align-items: center;
926 |     text-align: center;
927 |     box-shadow: 0 20px 40px rgba(0,0,0,0.4);
928 |     position: relative;
929 |     overflow: hidden;
930 |     animation: zoomIn 0.35s cubic-bezier(0.175, 0.885, 0.32, 1.275);
931 | }
932 |

```

```

933 | .active-marking-card::before {
934 |     content: '';
935 |     position: absolute;
936 |     top: 0; left: 0; right: 0; height: 4px;
937 |     background: linear-gradient(90deg, var(--primary), var(--accent));
938 | }
939 |
940 | .active-student-profile {
941 |     margin: 1.5rem 0 2rem;
942 | }
943 | .active-student-profile h4 { font-size: 1.8rem; font-family: var(--font-heading); margin-bottom: 0.5rem;
font-weight: 800; }
944 | .active-student-profile span { color: var(--text-muted); font-size: 0.95rem; }
945 |
946 | .marking-actions {
947 |     display: flex;
948 |     gap: 1rem;
949 |     width: 100%;
950 |     margin-bottom: 2rem;
951 | }
952 |
953 | .btn-absent, .btn-present {
954 |     flex: 1;
955 |     padding: 1rem;
956 |     border-radius: 16px;
957 |     border: none;
958 |     font-family: var(--font-heading);
959 |     font-weight: 800;
960 |     font-size: 1rem;
961 |     cursor: pointer;
962 |     display: flex;
963 |     flex-direction: column;
964 |     align-items: center;
965 |     gap: 0.5rem;
966 |     transition: var(--transition-bounce);
967 |     color: white;
968 | }
969 |
970 | .btn-absent {
971 |     background: linear-gradient(135deg, #ef4444, #be123c);
972 |     box-shadow: 0 4px 15px rgba(239, 68, 68, 0.25);
973 | }
974 | .btn-absent:hover { transform: translateY(-5px); box-shadow: 0 8px 25px rgba(239, 68, 68, 0.45); }
975 |
976 | .btn-present {
977 |     background: linear-gradient(135deg, #10b981, #047857);
978 |     box-shadow: 0 4px 15px rgba(16, 185, 129, 0.25);
979 | }
980 | .btn-present:hover { transform: translateY(-5px); box-shadow: 0 8px 25px rgba(16, 185, 129, 0.45); }
981 |
982 | .queue-progress-bar {
983 |     height: 6px;
984 |     background: rgba(255,255,255,0.08);
985 |     border-radius: 3px;
986 |     overflow: hidden;
987 | }
988 | .queue-progress-fill {
989 |     height: 100%;
990 |     background: var(--primary);
991 |     transition: width 0.4s cubic-bezier(0.25, 0.8, 0.25, 1);
992 | }
993 |
994 | .stack-item {
995 |     display: flex;
996 |     justify-content: space-between;
997 |     align-items: center;
998 |     padding: 0.8rem 1rem;
999 |     background: rgba(255,255,255,0.015);
1000 |     border: 1px solid var(--border-glass);
1001 |     border-radius: 10px;
1002 |     margin-bottom: 0.5rem;
1003 |     animation: slideDown 0.3s cubic-bezier(0.25, 0.8, 0.25, 1);
1004 | }

```

```

1005 |
1006 | .stack-item.item-present { border-left: 3px solid var(--success); }
1007 | .stack-item.item-absent { border-left: 3px solid var(--danger); }
1008 | .item-action { font-size: 0.75rem; font-weight: 800; text-transform: uppercase; }
1009 | .item-present .item-action { color: var(--success); }
1010 | .item-absent .item-action { color: var(--danger); }
1011 |
1012 | /* --- DSA Cards --- */
1013 | .dsa-grid {
1014 |     display: grid;
1015 |     grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
1016 |     gap: 1.5rem;
1017 | }
1018 |
1019 | .dsa-card {
1020 |     background: rgba(255,255,255,0.015);
1021 |     border: 1px solid var(--border-glass);
1022 |     border-radius: 16px;
1023 |     padding: 1.5rem;
1024 |     display: flex;
1025 |     gap: 1.25rem;
1026 |     transition: var(--transition-bounce);
1027 | }
1028 |
1029 | .dsa-card:hover {
1030 |     background: rgba(255,255,255,0.03);
1031 |     transform: translateY(-5px);
1032 |     border-color: rgba(255,255,255,0.08);
1033 |     box-shadow: 0 10px 30px rgba(0,0,0,0.3);
1034 | }
1035 |
1036 | .dsa-card-icon {
1037 |     width: 48px;
1038 |     height: 48px;
1039 |     border-radius: 12px;
1040 |     background: rgba(255, 255, 255, 0.04);
1041 |     display: flex;
1042 |     align-items: center;
1043 |     justify-content: center;
1044 |     flex-shrink: 0;
1045 | }
1046 |
1047 | .dsa-card-content h4 { font-family: var(--font-heading); font-size: 1.1rem; margin-bottom: 0.5rem; font-
weight: 700; }
1048 | .dsa-card-content p { font-size: 0.85rem; color: var(--text-muted); margin-bottom: 0.75rem; line-height:
1.5; }
1049 | .dsa-card-content small { font-size: 0.8rem; color: rgba(255,255,255,0.4); display: block; }
1050 | .dsa-card-content code { background: rgba(0,0,0,0.35); padding: 0.1rem 0.3rem; border-radius: 4px; color:
var(--accent); }
1051 |
1052 | /* --- Tab Buttons --- */
1053 | .tab-button-group {
1054 |     display: flex;
1055 |     background: rgba(0,0,0,0.25);
1056 |     border-radius: 12px;
1057 |     padding: 0.25rem;
1058 | }
1059 |
1060 | .tab-btn {
1061 |     padding: 0.5rem 1rem;
1062 |     border: none;
1063 |     background: transparent;
1064 |     color: var(--text-muted);
1065 |     font-weight: 700;
1066 |     font-size: 0.85rem;
1067 |     border-radius: 8px;
1068 |     cursor: pointer;
1069 |     transition: var(--transition-fast);
1070 | }
1071 |
1072 | .tab-btn.active {
1073 |     background: var(--primary);
1074 |     color: white;

```

```

1075 |     box-shadow: 0 2px 10px rgba(139, 92, 246, 0.3);
1076 | }
1077 |
1078 | /* --- Animations --- */
1079 | @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
1080 | @keyframes slideDown { from { opacity: 0; transform: translateY(-20px); } to { opacity: 1; transform:
translateY(0); } }
1081 | @keyframes slideUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform:
translateY(0); } }
1082 | @keyframes zoomIn { from { opacity: 0; transform: scale(0.95); } to { opacity: 1; transform: scale(1); } }
1083 | @keyframes pulseGlow { 0% { opacity: 0.4; transform: scale(0.97); } 100% { opacity: 0.8; transform:
scale(1.03); } }
1084 | @keyframes pulse { 0% { box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.4); } 70% { box-shadow: 0 0 0 10px
rgba(16, 185, 129, 0); } 100% { box-shadow: 0 0 0 0 rgba(16, 185, 129, 0); } }
1085 |
1086 | /* Scrollbar */
1087 | ::-webkit-scrollbar { width: 8px; height: 8px; }
1088 | ::-webkit-scrollbar-track { background: var(--bg-dark); }
1089 | ::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.08); border-radius: 4px; }
1090 | ::-webkit-scrollbar-thumb:hover { background: rgba(255,255,255,0.15); }

```

app.js

JavaScript — 1628 lines

```

1 | // Student Data Store Module (localStorage persistence + O(1) hash indexing)
2 | const StudentStore = (() => {
3 |     const STORAGE_KEY = 'attendance_system_students';
4 |     let studentsCache = [];
5 |     let rollIndex = {}; // rollNo (int) -> index in studentsCache array
6 |
7 |     // Rebuild the index map mapping rollNo -> index in studentsCache.
8 |     // Time Complexity: O(n) where n is the number of student records.
9 |     function rebuildRollIndex() {
10 |         rollIndex = {};
11 |         studentsCache.forEach((student, idx) => {
12 |             rollIndex[student.rollNo] = idx;
13 |         });
14 |     }
15 |
16 |     // Load students from localStorage
17 |     // Time Complexity: O(n) where n is the length of stored JSON.
18 |     function loadStudents() {
19 |         try {
20 |             const rawData = localStorage.getItem(STORAGE_KEY);
21 |             return rawData ? JSON.parse(rawData) : [];
22 |         } catch (e) {
23 |             console.error("Error loading students from localStorage:", e);
24 |             return [];
25 |         }
26 |     }
27 |
28 |     // Save students to localStorage
29 |     // Time Complexity: O(n) serialization time.
30 |     function saveStudents() {
31 |         try {
32 |             localStorage.setItem(STORAGE_KEY, JSON.stringify(studentsCache));
33 |         } catch (e) {
34 |             console.error("Error saving students to localStorage:", e);
35 |         }
36 |     }
37 |
38 |     // Initialize cache and index immediately on script evaluation
39 |     studentsCache = loadStudents();
40 |     if (studentsCache.length === 0) {
41 |         // Pre-populate with random/mock student details as requested
42 |         studentsCache = [
43 |             { rollNo: 101, name: "Alice Smith", department: "CSE", totalClasses: 40, attendedClasses: 35 },
44 |             { rollNo: 102, name: "Bob Jones", department: "ECE", totalClasses: 45, attendedClasses: 30 },
45 |             { rollNo: 103, name: "Charlie Brown", department: "ME", totalClasses: 30, attendedClasses: 29
46 |             },
47 |             { rollNo: 104, name: "Diana Prince", department: "CSE", totalClasses: 50, attendedClasses: 32
48 |             },
49 |             { rollNo: 105, name: "Ethan Hunt", department: "IT", totalClasses: 35, attendedClasses: 28 }
50 |         ];
51 |         saveStudents();
52 |     }
53 |     rebuildRollIndex();
54 |
55 |     return {
56 |         // Retrieve all student records (Time Complexity: O(1) to return reference, O(n) to return copy)
57 |         getAll: () => {
58 |             return studentsCache; // Return reference directly to allow in-place sorting
59 |         },
60 |         // Get a student record by Roll No in O(1) time
61 |         getByRollNo: (rollNo) => {
62 |             const idx = rollIndex[parseInt(rollNo, 10)];
63 |             return idx !== undefined ? studentsCache[idx] : null;
64 |         },
65 |         // Get student index in O(1) time

```

```

66 |     getIndexByRollNo: (rollNo) => {
67 |         const idx = rollIndex[parseInt(rollNo, 10)];
68 |         return idx !== undefined ? idx : -1;
69 |     },
70 |
71 |     // Add a new student record
72 |     // Time Complexity: O(1) average without I/O. O(n) with serialization.
73 |     addStudent: (student) => {
74 |         const rollNo = parseInt(student.rollNo, 10);
75 |         if (isNaN(rollNo)) {
76 |             throw new Error("Invalid Roll Number");
77 |         }
78 |         if (rollIndex[rollNo] !== undefined) {
79 |             throw new Error(`Student with Roll No ${rollNo} already exists.`);
80 |         }
81 |
82 |         const newStudent = {
83 |             rollNo: rollNo,
84 |             name: student.name.trim(),
85 |             department: student.department.trim(),
86 |             totalClasses: parseInt(student.totalClasses, 10) || 0,
87 |             attendedClasses: parseInt(student.attendedClasses, 10) || 0
88 |         };
89 |
90 |         studentsCache.push(newStudent);
91 |         rollIndex[rollNo] = studentsCache.length - 1;
92 |         saveStudents();
93 |         return newStudent;
94 |     },
95 |
96 |     // Delete student by Roll No
97 |     // Time Complexity: O(n) due to array shifting (splice) + map rebuild + serialization.
98 |     deleteStudent: (rollNo) => {
99 |         const targetRoll = parseInt(rollNo, 10);
100 |         const idx = rollIndex[targetRoll];
101 |         if (idx === undefined) {
102 |             return false;
103 |         }
104 |
105 |         studentsCache.splice(idx, 1);
106 |         rebuildRollIndex();
107 |         saveStudents();
108 |         return true;
109 |     },
110 |
111 |     // Update student details
112 |     // Time Complexity: O(1) lookup. O(n) file serialization.
113 |     updateStudent: (rollNo, data) => {
114 |         const targetRoll = parseInt(rollNo, 10);
115 |         const idx = rollIndex[targetRoll];
116 |         if (idx === undefined) {
117 |             return false;
118 |         }
119 |
120 |         studentsCache[idx] = {
121 |             ...studentsCache[idx],
122 |             name: data.name ? data.name.trim() : studentsCache[idx].name,
123 |             department: data.department ? data.department.trim() : studentsCache[idx].department,
124 |             totalClasses: data.totalClasses !== undefined ? parseInt(data.totalClasses, 10) :
studentsCache[idx].totalClasses,
125 |             attendedClasses: data.attendedClasses !== undefined ? parseInt(data.attendedClasses, 10) :
studentsCache[idx].attendedClasses
126 |         };
127 |
128 |         saveStudents();
129 |         return true;
130 |     },
131 |
132 |     // Rebuild index after sorting or mutation shifts
133 |     rebuildIndex: () => {
134 |         rebuildRollIndex();
135 |     }
136 | };

```

```

137 | });
138 |
139 | // Single-Page Application (SPA) Routing & Scaffold Initialization
140 | // Handled at the end of the file
141 |
142 |
143 | // Navigation / Section Toggle Routing
144 | function initNavigation() {
145 |     const menuItems = document.querySelectorAll('.menu-item');
146 |     const sections = document.querySelectorAll('.content-section');
147 |     const pageTitle = document.getElementById('page-title');
148 |     const pageSubtitle = document.getElementById('page-subtitle');
149 |
150 |     // Section Meta Data for Page Header updates
151 |     const sectionMeta = {
152 |         'dashboard': {
153 |             title: 'Dashboard',
154 |             subtitle: 'Welcome to the Attendance Management System dashboard'
155 |         },
156 |         'add-student': {
157 |             title: 'Add / Edit Student',
158 |             subtitle: 'Register a new student record into local memory store'
159 |         },
160 |         'search': {
161 |             title: 'Search Database',
162 |             subtitle: 'Locate student records and benchmark search algorithms'
163 |         },
164 |         'attendance': {
165 |             title: 'Mark Attendance',
166 |             subtitle: 'Batch process roll numbers using queue serialization with undo history'
167 |         },
168 |         'sorting': {
169 |             title: 'Sorting Algorithms',
170 |             subtitle: 'Sort student records using Bubble Sort or Merge Sort with complexity visualization'
171 |         },
172 |         'reports': {
173 |             title: 'Reports & Charts',
174 |             subtitle: 'View class metrics, deficiency lists, and charts'
175 |         },
176 |         'about': {
177 |             title: 'About & DSA Info',
178 |             subtitle: 'Detailed documentation of C++ Engine and Web Portal mapping for evaluators'
179 |         }
180 |     };
181 |
182 |     menuItems.forEach(item => {
183 |         item.addEventListener('click', () => {
184 |             const sectionName = item.getAttribute('data-section');
185 |             const targetSection = document.getElementById(`sec-${sectionName}`);
186 |
187 |             if (!targetSection) {
188 |                 console.error(`Target section 'sec-${sectionName}' not found.`);
189 |                 return;
190 |             }
191 |
192 |             // Remove active classes
193 |             menuItems.forEach(btn => btn.classList.remove('active'));
194 |             sections.forEach(sec => sec.classList.remove('active'));
195 |
196 |             // Add active classes to current item and section
197 |             item.classList.add('active');
198 |             targetSection.classList.add('active');
199 |
200 |             // Dynamic header details animation / update
201 |             if (sectionMeta[sectionName]) {
202 |                 pageTitle.textContent = sectionMeta[sectionName].title;
203 |                 pageSubtitle.textContent = sectionMeta[sectionName].subtitle;
204 |             }
205 |
206 |             if (sectionName === 'sorting') {
207 |                 renderVisualizerArray();
208 |             }
209 |         });

```

```

210 |         if (sectionName === 'reports') {
211 |             updateReportsStats();
212 |             renderReportsTable();
213 |             renderAttendanceChart();
214 |         }
215 |
216 |         console.log(`Navigated to section: ${sectionName}`);
217 |     });
218 | });
219 | }
220 |
221 | // Update high-level dashboard summaries from StudentStore
222 | function updateDashboardStats() {
223 |     const students = StudentStore.getAll();
224 |     const totalCount = students.length;
225 |     let avgAttendance = 0.0;
226 |     let lowAttendanceCount = 0;
227 |
228 |     if (totalCount > 0) {
229 |         let totalPct = 0;
230 |         students.forEach(s => {
231 |             const pct = s.totalClasses > 0 ? (s.attendedClasses / s.totalClasses) * 100 : 0;
232 |             totalPct += pct;
233 |             if (pct < 75.0) {
234 |                 lowAttendanceCount++;
235 |             }
236 |         });
237 |         avgAttendance = totalPct / totalCount;
238 |     }
239 |
240 |     document.getElementById('dash-stat-total').textContent = totalCount;
241 |     document.getElementById('dash-stat-avg').textContent = `${avgAttendance.toFixed(1)}%`;
242 |     document.getElementById('dash-stat-low').textContent = lowAttendanceCount;
243 | }
244 |
245 | // Global active edit session state
246 | let editStudentRoll = null;
247 |
248 | // Render the Directory list table in Add Student Section
249 | function renderDirectory() {
250 |     const tbody = document.getElementById('directory-tbody');
251 |     if (!tbody) return;
252 |
253 |     tbody.innerHTML = '';
254 |     const students = StudentStore.getAll();
255 |
256 |     if (students.length === 0) {
257 |         tbody.innerHTML = `
258 |             <tr>
259 |                 <td colspan="7" style="text-align: center; color: var(--text-muted); font-style: italic;
padding: 2rem;">
260 |                     No students registered in the database.
261 |                 </td>
262 |             </tr>
263 |         `;
264 |         return;
265 |     }
266 |
267 |     students.forEach(student => {
268 |         const pct = student.totalClasses > 0 ? (student.attendedClasses / student.totalClasses) * 100 : 0;
269 |         const statusClass = pct >= 75.0 ? 'badge-success' : 'badge-danger';
270 |
271 |         const tr = document.createElement('tr');
272 |         tr.innerHTML = `
273 |             <td><strong>#${student.rollNo}</strong></td>
274 |             <td>${student.name}</td>
275 |             <td><span style="background-color: rgba(255,255,255,0.05); padding: 0.2rem 0.5rem; border-
radius: 6px; font-size: 0.8rem;">${student.department}</span></td>
276 |             <td>${student.totalClasses}</td>
277 |             <td>${student.attendedClasses}</td>
278 |             <td><span class="badge ${statusClass}">${pct.toFixed(1)}%</span></td>
279 |             <td class="text-right">
280 |                 <div class="btn-action-group">

```

```

281 |                 <button class="btn-action btn-edit" title="Edit Student" data-roll="${student.rollNo}">
282 |                     <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><path d="M12 20h9"/><path d="M16.5 3.5a2.121
2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z"/></svg>
283 |                 </button>
284 |                 <button class="btn-action btn-delete" title="Delete Student" data-
roll="${student.rollNo}">
285 |                     <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><polyline points="3 6 5 6 21 6"/><path d="M19
6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"/><line x1="10" y1="11" x2="10"
y2="17"/><line x1="14" y1="11" x2="14" y2="17"/></svg>
286 |                 </button>
287 |             </div>
288 |         </td>
289 |     `;
290 |
291 |     // Wire Event listeners to Action Buttons
292 |     tr.querySelector('.btn-edit').addEventListener('click', () => {
293 |         enableEditMode(student.rollNo);
294 |     });
295 |
296 |     tr.querySelector('.btn-delete').addEventListener('click', () => {
297 |         if (confirm('Are you sure you want to delete student: ${student.name} (Roll:
${student.rollNo})?')) {
298 |             StudentStore.deleteStudent(student.rollNo);
299 |             renderDirectory();
300 |             updateDashboardStats();
301 |             if (editStudentRoll === student.rollNo) {
302 |                 cancelEditMode();
303 |             }
304 |         }
305 |     });
306 |
307 |     tbody.appendChild(tr);
308 | });
309 | }
310 |
311 | // Initialize Student Form Actions
312 | function initStudentForm() {
313 |     const form = document.getElementById('form-student');
314 |     const cancelBtn = document.getElementById('btn-cancel-edit');
315 |     if (!form) return;
316 |
317 |     form.addEventListener('submit', (e) => {
318 |         e.preventDefault();
319 |
320 |         const rollInput = document.getElementById('student-roll');
321 |         const nameInput = document.getElementById('student-name');
322 |         const deptInput = document.getElementById('student-dept');
323 |         const totalInput = document.getElementById('student-total');
324 |         const attendedInput = document.getElementById('student-attended');
325 |
326 |         const errorBanner = document.getElementById('form-error-banner');
327 |         const successBanner = document.getElementById('form-success-banner');
328 |
329 |         errorBanner.style.display = 'none';
330 |         successBanner.style.display = 'none';
331 |
332 |         const rollVal = parseInt(rollInput.value, 10);
333 |         const nameVal = nameInput.value.trim();
334 |         const deptVal = deptInput.value.trim();
335 |         const totalVal = parseInt(totalInput.value, 10);
336 |         const attendedVal = parseInt(attendedInput.value, 10);
337 |
338 |         // Validation constraints
339 |         if (isNaN(rollVal) || rollVal <= 0) {
340 |             showFormAlert(errorBanner, "Roll number must be a positive integer.");
341 |             return;
342 |         }
343 |
344 |         if (!nameVal || !deptVal) {
345 |             showFormAlert(errorBanner, "All fields are required. Spaces only are not allowed.");
346 |             return;

```

```

347 |     }
348 |
349 |     if (isNaN(totalVal) || totalVal < 0 || isNaN(attendedVal) || attendedVal < 0) {
350 |         showFormAlert(errorBanner, "Class held and attended counts must be non-negative integers.");
351 |         return;
352 |     }
353 |
354 |     if (attendedVal > totalVal) {
355 |         showFormAlert(errorBanner, "Attended lectures cannot exceed total classes conducted.");
356 |         return;
357 |     }
358 |
359 |     try {
360 |         if (editStudentRoll !== null) {
361 |             // Update mode
362 |             StudentStore.updateStudent(editStudentRoll, {
363 |                 name: nameVal,
364 |                 department: deptVal,
365 |                 totalClasses: totalVal,
366 |                 attendedClasses: attendedVal
367 |             });
368 |             showFormAlert(successBanner, "Student details updated successfully!");
369 |             cancelEditMode();
370 |         } else {
371 |             // Add Mode: check duplicates first
372 |             if (StudentStore.getByRollNo(rollVal) !== null) {
373 |                 showFormAlert(errorBanner, `Duplicate Roll: A student with roll #${rollVal} already
exists.`);
374 |                 return;
375 |             }
376 |
377 |             StudentStore.addStudent({
378 |                 rollNo: rollVal,
379 |                 name: nameVal,
380 |                 department: deptVal,
381 |                 totalClasses: totalVal,
382 |                 attendedClasses: attendedVal
383 |             });
384 |             showFormAlert(successBanner, "Student registered successfully!");
385 |             form.reset();
386 |         }
387 |
388 |         renderDirectory();
389 |         updateDashboardStats();
390 |
391 |     } catch (error) {
392 |         showFormAlert(errorBanner, error.message);
393 |     }
394 | });
395 |
396 | if (cancelBtn) {
397 |     cancelBtn.addEventListener('click', cancelEditMode);
398 | }
399 | }
400 |
401 | // Put Form into Edit Mode
402 | function enableEditMode(rollNo) {
403 |     const student = StudentStore.getByRollNo(rollNo);
404 |     if (!student) return;
405 |
406 |     editStudentRoll = rollNo;
407 |
408 |     const rollInput = document.getElementById('student-roll');
409 |     rollInput.value = student.rollNo;
410 |     rollInput.disabled = true; // Roll number key cannot be changed during edits
411 |
412 |     document.getElementById('student-name').value = student.name;
413 |     document.getElementById('student-dept').value = student.department;
414 |     document.getElementById('student-total').value = student.totalClasses;
415 |     document.getElementById('student-attended').value = student.attendedClasses;
416 |
417 |     document.getElementById('form-title').textContent = "Modify Student Details";

```

```

418 |     document.getElementById('form-desc').textContent = `Editing student card with Roll No
#${student.rollNo}`;
419 |     document.getElementById('btn-submit-student').textContent = "Save Changes";
420 |     document.getElementById('btn-cancel-edit').style.display = 'inline-flex';
421 |
422 |     document.querySelector('.card-form').scrollIntoView({ behavior: 'smooth' });
423 | }
424 |
425 | // Exit Edit Mode and clear form
426 | function cancelEditMode() {
427 |     editStudentRoll = null;
428 |     const form = document.getElementById('form-student');
429 |     if (form) form.reset();
430 |
431 |     const rollInput = document.getElementById('student-roll');
432 |     if (rollInput) rollInput.disabled = false;
433 |
434 |     document.getElementById('form-title').textContent = "Register New Student";
435 |     document.getElementById('form-desc').textContent = "Insert details to register a student record into
local memory store.";
436 |     document.getElementById('btn-submit-student').textContent = "Add Student";
437 |     document.getElementById('btn-cancel-edit').style.display = 'none';
438 | }
439 |
440 | // Display timed banners
441 | function showFormAlert(element, message) {
442 |     element.textContent = message;
443 |     element.style.display = 'block';
444 |     element.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
445 |     setTimeout(() => {
446 |         element.style.display = 'none';
447 |     }, 4000);
448 | }
449 |
450 | // Linear Search Implementation (O(N) sequential search)
451 | // Time Complexity: O(n) average/worst case.
452 | // Space Complexity: O(1) auxiliary space.
453 | function linearSearch(students, rollNo) {
454 |     let comparisons = 0;
455 |     for (let i = 0; i < students.length; ++i) {
456 |         comparisons++;
457 |         if (students[i].rollNo === rollNo) {
458 |             return { index: i, comparisons };
459 |         }
460 |     }
461 |     return { index: -1, comparisons };
462 | }
463 |
464 | // Binary Search Implementation (O(log N) divide-and-conquer search)
465 | // Time Complexity: O(log n) worst/average case. Assumes sorted dataset.
466 | // Space Complexity: O(1) auxiliary space.
467 | function binarySearch(students, rollNo) {
468 |     let comparisons = 0;
469 |     let low = 0;
470 |     let high = students.length - 1;
471 |
472 |     while (low <= high) {
473 |         comparisons++;
474 |         let mid = Math.floor(low + (high - low) / 2);
475 |         let midRoll = students[mid].rollNo;
476 |
477 |         if (midRoll === rollNo) {
478 |             return { index: mid, comparisons };
479 |         } else if (midRoll < rollNo) {
480 |             low = mid + 1;
481 |         } else {
482 |             high = mid - 1;
483 |         }
484 |     }
485 |     return { index: -1, comparisons };
486 | }
487 |
488 | // Initialize Search Configuration handlers

```

```

489 | function initSearchForm() {
490 |     const form = document.getElementById('form-search');
491 |     if (!form) return;
492 |
493 |     form.addEventListener('submit', (e) => {
494 |         e.preventDefault();
495 |
496 |         const rollInput = document.getElementById('search-roll-input');
497 |         const rollVal = parseInt(rollInput.value, 10);
498 |         const method = document.querySelector('input[name="search-method"]:checked').value;
499 |         const resultsArea = document.getElementById('search-results-area');
500 |         const timerBadge = document.getElementById('search-timer-badge');
501 |         const timerText = document.getElementById('search-time-taken');
502 |
503 |         if (isNaN(rollVal) || rollVal <= 0) {
504 |             resultsArea.innerHTML = `
505 |                 <div class="error-banner">Please enter a valid positive roll number to search.</div>
506 |             `;
507 |             if (timerBadge) timerBadge.style.display = 'none';
508 |             return;
509 |         }
510 |
511 |         const students = StudentStore.getAll();
512 |         if (students.length === 0) {
513 |             resultsArea.innerHTML = `
514 |                 <div class="error-banner">No student records are currently available in the database.</div>
515 |             `;
516 |             if (timerBadge) timerBadge.style.display = 'none';
517 |             return;
518 |         }
519 |
520 |         let warningHtml = '';
521 |         let isSorted = true;
522 |         // Verify sorted property
523 |         for (let i = 1; i < students.length; ++i) {
524 |             if (students[i].rollNo < students[i - 1].rollNo) {
525 |                 isSorted = false;
526 |                 break;
527 |             }
528 |         }
529 |
530 |         if (method === 'binary' && !isSorted) {
531 |             // Sort database dynamically using native Array.prototype.sort (Timsort/Merge Sort derivative)
532 |             students.sort((a, b) => a.rollNo - b.rollNo);
533 |             StudentStore.rebuildIndex();
534 |             renderDirectory(); // refresh directory tables to show sorted order
535 |             warningHtml = `
536 |                 <div class="search-warning-banner">
537 |                     <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71
3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/><line x1="12" y1="9" x2="12" y2="13"/><line x1="12"
y1="17" x2="12.01" y2="17"/></svg>
538 |                     <span><strong>Pre-sort Triggered:</strong> Database unsorted. Sorted Roll No ascending
before running Binary Search.</span>
539 |                 </div>
540 |             `;
541 |         }
542 |
543 |         const t0 = performance.now();
544 |         let result;
545 |         if (method === 'linear') {
546 |             result = linearSearch(students, rollVal);
547 |         } else {
548 |             result = binarySearch(students, rollVal);
549 |         }
550 |         const t1 = performance.now();
551 |         const duration = t1 - t0;
552 |
553 |         if (timerBadge && timerText) {
554 |             timerBadge.style.display = 'inline-flex';
555 |             timerText.textContent = `${duration.toFixed(3)} ms`;
556 |         }
557 |     }

```

```

558 |     renderSearchResults(result, rollVal, method, warningHtml);
559 | });
560 | }
561 |
562 | // Render search details and comparison statistics
563 | function renderSearchResults(result, rollNo, method, warningHtml) {
564 |     const resultsArea = document.getElementById('search-results-area');
565 |     if (!resultsArea) return;
566 |
567 |     if (result.index === -1) {
568 |         resultsArea.innerHTML = `
569 |             <div style="display: flex; flex-direction: column; gap: 1.5rem;">
570 |                 ${warningHtml}
571 |                 <div class="error-banner">
572 |                     Student with Roll No #${rollNo} was not found in the database.
573 |                 </div>
574 |                 <div class="benchmark-board">
575 |                     <div class="benchmark-card metric-highlight">
576 |                         <span class="bench-title">Comparisons Made</span>
577 |                         <span class="bench-value">${result.comparisons}</span>
578 |                         <span class="bench-desc">Attempts to locate Roll No #${rollNo}</span>
579 |                     </div>
580 |                     <div class="benchmark-card">
581 |                         <span class="bench-title">Complexity Class</span>
582 |                         <span class="bench-value">${method === 'linear' ? 'O(N)' : 'O(log N)'}</span>
583 |                         <span class="bench-desc">${method === 'linear' ? 'Full sequential scanning
required' : 'Split-half lookup efficiency'}</span>
584 |                     </div>
585 |                 </div>
586 |             </div>
587 |         `;
588 |         return;
589 |     }
590 |
591 |     const students = StudentStore.getAll();
592 |     const student = students[result.index];
593 |     const pct = student.totalClasses > 0 ? (student.attendedClasses / student.totalClasses) * 100 : 0;
594 |     const statusClass = pct >= 75.0 ? 'badge-success' : 'badge-danger';
595 |     const statusText = pct >= 75.0 ? 'Eligible' : 'Not Eligible';
596 |
597 |     resultsArea.innerHTML = `
598 |         <div class="search-result-card">
599 |             ${warningHtml}
600 |
601 |             <div class="result-main-info">
602 |                 <div class="result-meta-name">
603 |                     <h4>${student.name}</h4>
604 |                     <span>Dept: ${student.department} &bull; Roll: #${student.rollNo}</span>
605 |                 </div>
606 |                 <div class="result-pct-badge">
607 |                     <span class="result-pct-value">${pct.toFixed(1)}%</span>
608 |                     <div style="margin-top: 0.25rem;"><span class="badge
${statusClass}>${statusText}</span></div>
609 |                 </div>
610 |             </div>
611 |
612 |             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">
613 |                 <div style="background-color: rgba(255,255,255,0.01); border: 1px solid var(--border-
color); border-radius: 10px; padding: 1rem; text-align: center;">
614 |                     <span style="font-size: 0.75rem; text-transform: uppercase; color: var(--text-muted);
font-weight:600; display:block; margin-bottom:0.25rem;">Lectures Attended</span>
615 |                     <strong style="font-size: 1.25rem; color: white;">${student.attendedClasses} /
${student.totalClasses}</strong>
616 |                 </div>
617 |                 <div style="background-color: rgba(255,255,255,0.01); border: 1px solid var(--border-
color); border-radius: 10px; padding: 1rem; text-align: center;">
618 |                     <span style="font-size: 0.75rem; text-transform: uppercase; color: var(--text-muted);
font-weight:600; display:block; margin-bottom:0.25rem;">Vector Position</span>
619 |                     <strong style="font-size: 1.25rem; color: white;">Index ${result.index}</strong>
620 |                 </div>
621 |             </div>
622 |
623 |             <div class="benchmark-board">

```

```

624 |         <div class="benchmark-card metric-highlight">
625 |             <span class="bench-title">Comparisons Made</span>
626 |             <span class="bench-value">${result.comparisons}</span>
627 |             <span class="bench-desc">Checks to find student roll #${student.rollNo}</span>
628 |         </div>
629 |         <div class="benchmark-card">
630 |             <span class="bench-title">Algorithmic Complexity</span>
631 |             <span class="bench-value">${method === 'linear' ? 'O(N)' : 'O(log N)'}</span>
632 |             <span class="bench-desc">${method === 'linear' ? 'Worse scalability' : 'Vastly superior
scalability'}</span>
633 |         </div>
634 |     </div>
635 | </div>
636 | `;
637 | }
638 |
639 | // Get current sorting speed delay (ms)
640 | function getAnimationSpeed() {
641 |     const slider = document.getElementById('sort-speed-range');
642 |     return slider ? parseInt(slider.value, 10) : 400;
643 | }
644 |
645 | // Get the criteria value for comparing
646 | function getSortValue(student, criteria) {
647 |     if (criteria === 'rollNo') {
648 |         return student.rollNo;
649 |     } else if (criteria === 'name') {
650 |         return student.name.toLowerCase();
651 |     } else { // attendancePct
652 |         return student.totalClasses > 0 ? (student.attendedClasses / student.totalClasses) * 100 : 0;
653 |     }
654 | }
655 |
656 | // Compare two values based on criteria (returns true if val1 > val2 - for ascending)
657 | function compareValues(val1, val2) {
658 |     return val1 > val2;
659 | }
660 |
661 | // Render the visualizer state
662 | function renderVisualizerArray(activeIndices = {}) {
663 |     const container = document.getElementById('visualizer-array-wrapper');
664 |     if (!container) return;
665 |
666 |     container.innerHTML = '';
667 |     const students = StudentStore.getAll();
668 |     const criteria = document.getElementById('sort-criteria-select').value;
669 |
670 |     if (students.length === 0) {
671 |         container.innerHTML = `
672 |             <div class="scaffold-placeholder">
673 |                 Register students first to visualize sorting algorithms.
674 |             </div>
675 |         `;
676 |         return;
677 |     }
678 |
679 |     // Determine max values to scale bars
680 |     let maxVal = 1;
681 |     if (criteria === 'rollNo') {
682 |         maxVal = Math.max(...students.map(s => s.rollNo), 1);
683 |     } else if (criteria === 'name') {
684 |         maxVal = 26; // Letter map scale A-Z
685 |     } else {
686 |         maxVal = 100; // Percentage scale
687 |     }
688 |
689 |     students.forEach((student, idx) => {
690 |         let displayVal = '';
691 |         let barPct = 0;
692 |
693 |         const criteriaVal = getSortValue(student, criteria);
694 |         if (criteria === 'rollNo') {
695 |             displayVal = `#${student.rollNo}`;

```

```

696 |         barPct = (student.rollNo / maxVal) * 100;
697 |     } else if (criteria === 'name') {
698 |         displayVal = student.name;
699 |         const code = student.name.toUpperCase().charCodeAt(0) - 65;
700 |         barPct = Math.max(((code >= 0 && code < 26 ? code : 13) / 25) * 100, 10);
701 |     } else {
702 |         displayVal = `${criteriaVal.toFixed(1)}%`;
703 |         barPct = criteriaVal;
704 |     }
705 |
706 |     const activeClass = activeIndices[idx] || '';
707 |
708 |     const card = document.createElement('div');
709 |     card.className = `visualizer-element-card ${activeClass}`;
710 |     card.innerHTML = `
711 |         <span class="element-index">Index ${idx}</span>
712 |         <div class="element-meta">
713 |             <strong>${student.name}</strong>
714 |             <span>Roll: #${student.rollNo} &bull; Dept: ${student.department}</span>
715 |         </div>
716 |         <div class="element-bar-container">
717 |             <div class="element-bar" style="width: ${barPct}%;"></div>
718 |         </div>
719 |         <span class="element-value">${displayVal}</span>
720 |     `;
721 |     container.appendChild(card);
722 | });
723 | }
724 |
725 | // Disable/enable controls during sorting
726 | function toggleVisualizerControls(disabled) {
727 |     const buttons = document.querySelectorAll('.btn-sort-trigger');
728 |     const select = document.getElementById('sort-criteria-select');
729 |     const slider = document.getElementById('sort-speed-range');
730 |     const menuButtons = document.querySelectorAll('.menu-item');
731 |
732 |     buttons.forEach(btn => btn.disabled = disabled);
733 |     if (select) select.disabled = disabled;
734 |     if (slider) slider.disabled = disabled;
735 |     menuButtons.forEach(btn => {
736 |         if (disabled) btn.style.pointerEvents = 'none';
737 |         else btn.style.pointerEvents = 'auto';
738 |     });
739 | }
740 |
741 | // Bubble Sort Visualizer
742 | async function bubbleSortVisual(criteria) {
743 |     const students = StudentStore.getAll();
744 |     const n = students.length;
745 |     const delayVal = () => new Promise(res => setTimeout(res, getAnimationSpeed()));
746 |
747 |     const statusBoard = document.getElementById('visualizer-status-board');
748 |     const statusText = document.getElementById('sort-status-text');
749 |     const stepInfo = document.getElementById('sort-step-info');
750 |     const timerDisplay = document.getElementById('sort-timer-display');
751 |
752 |     statusBoard.style.display = 'block';
753 |     statusText.textContent = "Running Bubble Sort (O(N²))";
754 |     toggleVisualizerControls(true);
755 |
756 |     const startTime = performance.now();
757 |     const updateTimer = () => {
758 |         if (timerDisplay) {
759 |             timerDisplay.textContent = `${(performance.now() - startTime).toFixed(2)} ms`;
760 |         }
761 |     };
762 |
763 |     let swaps = 0;
764 |     let comparisons = 0;
765 |
766 |     for (let i = 0; i < n - 1; i++) {
767 |         let swapped = false;
768 |         stepInfo.innerHTML = `<strong>Pass ${i + 1}</strong> Scanning index 0 to ${n - i - 2}.`;

```

```

769 |
770 |     for (let j = 0; j < n - i - 1; j++) {
771 |         comparisons++;
772 |         updateTimer();
773 |         renderVisualizerArray({ [j]: 'comparing', [j + 1]: 'comparing' });
774 |         stepInfo.innerHTML = `Comparing: Index ${j} and Index ${j + 1}.<br>Comparisons: ${comparisons}
&bull; Swaps: ${swaps}`;
775 |         await delayVal();
776 |
777 |         const val1 = getSortValue(students[j], criteria);
778 |         const val2 = getSortValue(students[j + 1], criteria);
779 |
780 |         if (compareValues(val1, val2)) {
781 |             swaps++;
782 |             swapped = true;
783 |             updateTimer();
784 |             renderVisualizerArray({ [j]: 'swapping', [j + 1]: 'swapping' });
785 |             stepInfo.innerHTML = `Swap triggered: Index ${j} and Index ${j + 1}.<br>Comparisons:
${comparisons} &bull; Swaps: ${swaps}`;
786 |             await delayVal();
787 |
788 |             // Swap in memory reference
789 |             const temp = students[j];
790 |             students[j] = students[j + 1];
791 |             students[j + 1] = temp;
792 |
793 |             updateTimer();
794 |             renderVisualizerArray({ [j]: 'swapping', [j + 1]: 'swapping' });
795 |             await delayVal();
796 |         }
797 |     }
798 |
799 |     if (!swapped) {
800 |         stepInfo.innerHTML = `Early sorting optimization: No elements swapped in Pass ${i+1}. Database
is sorted!`;
801 |         break;
802 |     }
803 | }
804 |
805 | const elapsed = performance.now() - startTime;
806 | if (timerDisplay) timerDisplay.textContent = `${elapsed.toFixed(2)} ms`;
807 |
808 | statusText.textContent = "Sorted Successfully!";
809 | stepInfo.innerHTML = `<strong>Completed:</strong> Bubble Sort completed.<br>Total Comparisons:
${comparisons} &bull; Total Swaps: ${swaps}`;
810 |
811 | // Save state, rebuild indexing maps, update UI tables
812 | StudentStore.rebuildIndex();
813 | renderDirectory();
814 | updateDashboardStats();
815 |
816 | // Highlighting
817 | const allSorted = {};
818 | for (let i = 0; i < n; i++) allSorted[i] = 'sorted-highlight';
819 | renderVisualizerArray(allSorted);
820 |
821 | await new Promise(res => setTimeout(res, 1500));
822 | renderVisualizerArray();
823 | toggleVisualizerControls(false);
824 | }
825 |
826 | // Recursive Merge Sort Visualizer
827 | async function mergeSortVisual(students, left, right, criteria, stats) {
828 |     if (left >= right) return;
829 |
830 |     const mid = Math.floor(left + (right - left) / 2);
831 |
832 |     await mergeSortVisual(students, left, mid, criteria, stats);
833 |     await mergeSortVisual(students, mid + 1, right, criteria, stats);
834 |     await mergeVisual(students, left, mid, right, criteria, stats);
835 | }
836 |
837 | // Merge Combine Step Animator

```

```

838 | async function mergeVisual(students, left, mid, right, criteria, stats) {
839 |     const delayVal = () => new Promise(res => setTimeout(res, getAnimationSpeed()));
840 |     const stepInfo = document.getElementById('sort-step-info');
841 |     const timerDisplay = document.getElementById('sort-timer-display');
842 |
843 |     const updateTimer = () => {
844 |         if (timerDisplay && stats.startTime) {
845 |             timerDisplay.textContent = `${(performance.now() - stats.startTime).toFixed(2)} ms`;
846 |         }
847 |     };
848 |
849 |     let n1 = mid - left + 1;
850 |     let n2 = right - mid;
851 |
852 |     let L = [];
853 |     let R = [];
854 |
855 |     for (let i = 0; i < n1; i++) L.push(students[left + i]);
856 |     for (let j = 0; j < n2; j++) R.push(students[mid + 1 + j]);
857 |
858 |     let i = 0, j = 0, k = left;
859 |
860 |     while (i < n1 && j < n2) {
861 |         stats.comparisons++;
862 |         updateTimer();
863 |         renderVisualizerArray({ [left + i]: 'comparing', [mid + 1 + j]: 'comparing' });
864 |         stepInfo.innerHTML = `Merging subarrays [${left}..${mid}] and [${mid+1}..${right}].<br>Comparing:
Index ${left+i} and Index ${mid+1+j}.<br>Comparisons: ${stats.comparisons}`;
865 |         await delayVal();
866 |
867 |         const val1 = getSortValue(L[i], criteria);
868 |         const val2 = getSortValue(R[j], criteria);
869 |
870 |         // Standard comparison check for sorting stability
871 |         if (val1 <= val2) {
872 |             students[k] = L[i];
873 |             i++;
874 |         } else {
875 |             students[k] = R[j];
876 |             j++;
877 |         }
878 |
879 |         updateTimer();
880 |         renderVisualizerArray({ [k]: 'swapping' });
881 |         await delayVal();
882 |         k++;
883 |     }
884 |
885 |     while (i < n1) {
886 |         students[k] = L[i];
887 |         updateTimer();
888 |         renderVisualizerArray({ [k]: 'swapping' });
889 |         await delayVal();
890 |         i++;
891 |         k++;
892 |     }
893 |
894 |     while (j < n2) {
895 |         students[k] = R[j];
896 |         updateTimer();
897 |         renderVisualizerArray({ [k]: 'swapping' });
898 |         await delayVal();
899 |         j++;
900 |         k++;
901 |     }
902 | }
903 |
904 | // Merge Sort Trigger
905 | async function triggerMergeSort(criteria) {
906 |     const students = StudentStore.getAll();
907 |     const n = students.length;
908 |
909 |     const statusBoard = document.getElementById('visualizer-status-board');

```

```

910 |     const statusText = document.getElementById('sort-status-text');
911 |     const stepInfo = document.getElementById('sort-step-info');
912 |     const timerDisplay = document.getElementById('sort-timer-display');
913 |
914 |     statusBoard.style.display = 'block';
915 |     statusText.textContent = "Running Merge Sort ( $O(N \log N)$ )";
916 |     toggleVisualizerControls(true);
917 |
918 |     const startTime = performance.now();
919 |     const stats = { comparisons: 0, startTime };
920 |
921 |     stepInfo.innerHTML = `Dividing array recursively and merging...`;
922 |     await mergeSortVisual(students, 0, n - 1, criteria, stats);
923 |
924 |     const elapsed = performance.now() - startTime;
925 |     if (timerDisplay) timerDisplay.textContent = `${elapsed.toFixed(2)} ms`;
926 |
927 |     statusText.textContent = "Sorted Successfully!";
928 |     stepInfo.innerHTML = `<strong>Completed:</strong> Merge Sort completed.<br>Total Comparisons:
    ${stats.comparisons}`;
929 |
930 |     // Save state, rebuild indexing maps, update UI tables
931 |     StudentStore.rebuildIndex();
932 |     renderDirectory();
933 |     updateDashboardStats();
934 |
935 |     // Highlighting
936 |     const allSorted = {};
937 |     for (let i = 0; i < n; i++) allSorted[i] = 'sorted-highlight';
938 |     renderVisualizerArray(allSorted);
939 |
940 |     await new Promise(res => setTimeout(res, 1500));
941 |     renderVisualizerArray();
942 |     toggleVisualizerControls(false);
943 | }
944 |
945 | // Initialize Sorting Visualizer Configuration Bindings
946 | function initSortingSection() {
947 |     const speedSlider = document.getElementById('sort-speed-range');
948 |     const speedLabel = document.getElementById('speed-label');
949 |     const bubbleBtn = document.getElementById('btn-trigger-bubble');
950 |     const mergeBtn = document.getElementById('btn-trigger-merge');
951 |     const select = document.getElementById('sort-criteria-select');
952 |
953 |     if (speedSlider) {
954 |         speedSlider.addEventListener('input', () => {
955 |             speedLabel.textContent = `${speedSlider.value}ms delay`;
956 |         });
957 |     }
958 |
959 |     if (bubbleBtn) {
960 |         bubbleBtn.addEventListener('click', () => {
961 |             const criteria = select.value;
962 |             bubbleSortVisual(criteria);
963 |         });
964 |     }
965 |
966 |     if (mergeBtn) {
967 |         mergeBtn.addEventListener('click', () => {
968 |             const criteria = select.value;
969 |             triggerMergeSort(criteria);
970 |         });
971 |     }
972 |
973 |     if (select) {
974 |         select.addEventListener('change', () => {
975 |             renderVisualizerArray();
976 |         });
977 |     }
978 | }
979 |
980 | // Attendance Queue & Stack (Undo) State management
981 | let attendanceQueue = [];

```

```

982 | let attendanceTotalItems = 0;
983 | let attendanceStack = []; // stores { rollNo, name, status }
984 |
985 | // Initialize Attendance Session handlers
986 | function initAttendanceSection() {
987 |     const initForm = document.getElementById('form-attendance-init');
988 |     const rollsInput = document.getElementById('attendance-rolls-input');
989 |     const undoBtn = document.getElementById('btn-undo-attendance');
990 |
991 |     if (initForm) {
992 |         initForm.addEventListener('submit', (e) => {
993 |             e.preventDefault();
994 |
995 |             const rawInput = rollsInput.value;
996 |             // Parse comma-separated or space-separated tokens
997 |             const rolls = rawInput
998 |                 .split(/[\s,]+/)
999 |                 .map(r => parseInt(r.trim(), 10))
1000 |                 .filter(r => !isNaN(r) && r > 0);
1001 |
1002 |             if (rolls.length === 0) {
1003 |                 alert("Please enter at least one valid positive Roll Number.");
1004 |                 return;
1005 |             }
1006 |
1007 |             // Filter rolls to ensure they exist in our database
1008 |             const validRolls = rolls.filter(roll => StudentStore.getByRollNo(roll) !== null);
1009 |
1010 |             if (validRolls.length === 0) {
1011 |                 alert("None of the entered Roll Numbers correspond to registered students.");
1012 |                 return;
1013 |             }
1014 |
1015 |             if (validRolls.length < rolls.length) {
1016 |                 alert(`Loaded ${validRolls.length} of ${rolls.length} entered roll numbers. (Unregistered
students skipped).`);
1017 |             }
1018 |
1019 |             // Initialize FIFO queue
1020 |             attendanceQueue = validRolls;
1021 |             attendanceTotalItems = validRolls.length;
1022 |             attendanceStack = []; // reset stack for new session
1023 |
1024 |             document.getElementById('queue-preview-box').style.display = 'block';
1025 |             renderQueuePreview();
1026 |             showNextMarkingCard();
1027 |             renderUndoStackList();
1028 |
1029 |             rollsInput.value = ''; // Clear input area
1030 |         });
1031 |     }
1032 |
1033 |     if (undoBtn) {
1034 |         undoBtn.addEventListener('click', () => {
1035 |             undoLastMarking();
1036 |         });
1037 |     }
1038 | }
1039 |
1040 | // Render FIFO preview pills
1041 | function renderQueuePreview() {
1042 |     const previewContainer = document.getElementById('queue-preview-pills');
1043 |     if (!previewContainer) return;
1044 |
1045 |     previewContainer.innerHTML = '';
1046 |     attendanceQueue.forEach((roll, idx) => {
1047 |         const student = StudentStore.getByRollNo(roll);
1048 |         const nameShort = student ? student.name.split(' ')[0] : 'Student';
1049 |         const isCurrent = idx === 0;
1050 |
1051 |         const pill = document.createElement('div');
1052 |         pill.className = `queue-pill ${isCurrent ? 'current' : ''}`;
1053 |         pill.innerHTML = `

```

```

1054 |         <span>#${roll}</span>
1055 |         <small style="opacity: 0.8; font-weight: 500;">(<small>${nameShort}</small>)</small>
1056 |     `;
1057 |     previewContainer.appendChild(pill);
1058 | });
1059 | }
1060 |
1061 | // Render dynamic current card or empty state
1062 | function showNextMarkingCard() {
1063 |     const markingArea = document.getElementById('attendance-marking-area');
1064 |     if (!markingArea) return;
1065 |
1066 |     if (attendanceQueue.length === 0) {
1067 |         markingArea.innerHTML = `
1068 |             <div class="empty-attendance-state" style="display: flex; flex-direction: column; align-items:
center; justify-content: center; gap: 1rem; color: var(--text-muted); text-align: center; animation: fadeIn 0.3s
ease;">
1069 |                 <svg width="48" height="48" viewBox="0 0 24 24" fill="none" stroke="var(--color-success)"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 11.08V12a10 10 0 1 1 -5.93-
9.14"/><polyline points="22 4 12 14.01 9 11.01"/></svg>
1070 |                 <strong>Marking Session Completed</strong>
1071 |                 <p style="font-size: 0.85rem; max-width: 320px;">All queue roll numbers have been
processed. Dashboard and reports are synchronized.</p>
1072 |             </div>
1073 |         `;
1074 |         document.getElementById('queue-preview-box').style.display = 'none';
1075 |         return;
1076 |     }
1077 |
1078 |     const currentRoll = attendanceQueue[0];
1079 |     const student = StudentStore.getByRollNo(currentRoll);
1080 |     if (!student) {
1081 |         // Safe check: student was deleted during session
1082 |         attendanceQueue.shift();
1083 |         showNextMarkingCard();
1084 |         return;
1085 |     }
1086 |
1087 |     const progressPct = ((attendanceTotalItems - attendanceQueue.length) / attendanceTotalItems) * 100;
1088 |
1089 |     markingArea.innerHTML = `
1090 |         <div class="active-marking-card">
1091 |             <span style="font-size: 0.8rem; background-color: var(--border-color); padding: 0.25rem 0.6rem;
border-radius: 20px; font-weight: 600; color: var(--text-muted);">
1092 |                 Up Next (${attendanceQueue.length} remaining)
1093 |             </span>
1094 |
1095 |             <div class="active-student-profile">
1096 |                 <h4>${student.name}</h4>
1097 |                 <span>Roll: #${student.rollNo} &bull; Dept: ${student.department}</span>
1098 |             </div>
1099 |
1100 |             <div class="marking-actions">
1101 |                 <button id="btn-mark-absent" class="btn btn-absent">
1102 |                     <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><line x1="18" y1="6" x2="6" y2="18"/><line
x1="6" y1="6" x2="18" y2="18"/></svg>
1103 |                     Absent (A)
1104 |                 </button>
1105 |                 <button id="btn-mark-present" class="btn btn-present">
1106 |                     <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><polyline points="20 6 9 17 4 12"/></svg>
1107 |                     Present (P)
1108 |                 </button>
1109 |             </div>
1110 |
1111 |             <div style="width: 100%;">
1112 |                 <div style="display: flex; justify-content: space-between; font-size: 0.75rem; color: var(-
-text-muted); margin-bottom: 0.35rem;">
1113 |                     <span>Session Progress</span>
1114 |                     <span>${Math.round(progressPct)}%</span>
1115 |                 </div>
1116 |                 <div class="queue-progress-bar">

```

```

1117 |         <div class="queue-progress-fill" style="width: ${progressPct}%;"></div>
1118 |     </div>
1119 | </div>
1120 | </div>
1121 | `;
1122 |
1123 | // Bind mark actions
1124 | document.getElementById('btn-mark-present').addEventListener('click', () => {
1125 |     executeMarking(currentRoll, 'Present');
1126 | });
1127 |
1128 | document.getElementById('btn-mark-absent').addEventListener('click', () => {
1129 |     executeMarking(currentRoll, 'Absent');
1130 | });
1131 | }
1132 |
1133 | // Process single roll marking
1134 | function executeMarking(rollNo, status) {
1135 |     const student = StudentStore.getByRollNo(rollNo);
1136 |     if (!student) return;
1137 |
1138 |     // Mutate state in StudentStore
1139 |     const newTotal = student.totalClasses + 1;
1140 |     const newAttended = student.attendedClasses + (status === 'Present' ? 1 : 0);
1141 |
1142 |     StudentStore.updateStudent(rollNo, {
1143 |         totalClasses: newTotal,
1144 |         attendedClasses: newAttended
1145 |     });
1146 |
1147 |     // Record action onto stack
1148 |     attendanceStack.push({
1149 |         rollNo: rollNo,
1150 |         name: student.name,
1151 |         status: status
1152 |     });
1153 |
1154 |     // Dequeue (FIFO)
1155 |     attendanceQueue.shift();
1156 |
1157 |     // Re-render components
1158 |     renderQueuePreview();
1159 |     showNextMarkingCard();
1160 |     renderUndoStackList();
1161 |
1162 |     // Sync other directories
1163 |     renderDirectory();
1164 |     updateDashboardStats();
1165 | }
1166 |
1167 | // Revert last marked action (Stack LIFO)
1168 | function undoLastMarking() {
1169 |     if (attendanceStack.length === 0) return;
1170 |
1171 |     // Pop off stack (LIFO)
1172 |     const lastAction = attendanceStack.pop();
1173 |     const student = StudentStore.getByRollNo(lastAction.rollNo);
1174 |
1175 |     if (student) {
1176 |         const currentTotal = student.totalClasses;
1177 |         const currentAttended = student.attendedClasses;
1178 |
1179 |         // Revert counters (safely boundaries)
1180 |         const undoneTotal = currentTotal > 0 ? currentTotal - 1 : 0;
1181 |         const undoneAttended = (lastAction.status === 'Present' && currentAttended > 0) ? currentAttended -
1 : currentAttended;
1182 |
1183 |         StudentStore.updateStudent(lastAction.rollNo, {
1184 |             totalClasses: undoneTotal,
1185 |             attendedClasses: undoneAttended
1186 |         });
1187 |     }
1188 | }

```

```

1189 | // Prepend reverted roll back to front of FIFO queue
1190 | attendanceQueue.unshift(lastAction.rollNo);
1191 |
1192 | // Sync elements
1193 | renderQueuePreview();
1194 | showNextMarkingCard();
1195 | renderUndoStackList();
1196 |
1197 | // Sync other tabs
1198 | renderDirectory();
1199 | updateDashboardStats();
1200 | }
1201 |
1202 | // Render LIFO log entries
1203 | function renderUndoStackList() {
1204 |     const container = document.getElementById('undo-stack-preview');
1205 |     const undoBtn = document.getElementById('btn-undo-attendance');
1206 |     if (!container || !undoBtn) return;
1207 |
1208 |     if (attendanceStack.length === 0) {
1209 |         container.innerHTML = `
1210 |             <div class="empty-stack-state" style="text-align: center; color: var(--text-muted); font-size:
0.85rem; font-style: italic; padding: 2rem 0;">
1211 |                 Stack is empty. No marks to undo.
1212 |             </div>
1213 |         `;
1214 |         undoBtn.disabled = true;
1215 |         return;
1216 |     }
1217 |
1218 |     undoBtn.disabled = false;
1219 |     container.innerHTML = '';
1220 |
1221 |     // Render stack from top to bottom (latest first)
1222 |     for (let idx = attendanceStack.length - 1; idx >= 0; idx--) {
1223 |         const item = attendanceStack[idx];
1224 |         const log = document.createElement('div');
1225 |         log.className = `stack-item ${item.status === 'Present' ? 'item-present' : 'item-absent'}`;
1226 |         log.innerHTML = `
1227 |             <div class="item-meta">
1228 |                 <strong>#${item.rollNo}</strong> - ${item.name}
1229 |             </div>
1230 |             <div class="item-action">${item.status}</div>
1231 |         `;
1232 |         container.appendChild(log);
1233 |     }
1234 | }
1235 |
1236 | // Reports Tab & Chart.js instances
1237 | let activeReportsTab = 'all'; // 'all' or 'deficient'
1238 | let attendanceChartInstance = null;
1239 |
1240 | // Initialize Reports Tab listeners
1241 | function initReportsSection() {
1242 |     const tabAll = document.getElementById('btn-tab-all-records');
1243 |     const tabDeficient = document.getElementById('btn-tab-deficient-records');
1244 |
1245 |     if (tabAll && tabDeficient) {
1246 |         tabAll.addEventListener('click', () => {
1247 |             tabAll.classList.add('active');
1248 |             tabDeficient.classList.remove('active');
1249 |             activeReportsTab = 'all';
1250 |             renderReportsTable();
1251 |         });
1252 |
1253 |         tabDeficient.addEventListener('click', () => {
1254 |             tabDeficient.classList.add('active');
1255 |             tabAll.classList.remove('active');
1256 |             activeReportsTab = 'deficient';
1257 |             renderReportsTable();
1258 |         });
1259 |     }
1260 | }

```

```

1261 |
1262 | // Update stats cards in Reports tab (mirrors dashboard stats)
1263 | function updateReportsStats() {
1264 |     const students = StudentStore.getAll();
1265 |     const totalCount = students.length;
1266 |     let avgAttendance = 0.0;
1267 |     let lowAttendanceCount = 0;
1268 |
1269 |     if (totalCount > 0) {
1270 |         let totalPct = 0;
1271 |         students.forEach(s => {
1272 |             const pct = s.totalClasses > 0 ? (s.attendedClasses / s.totalClasses) * 100 : 0;
1273 |             totalPct += pct;
1274 |             if (pct < 75.0) {
1275 |                 lowAttendanceCount++;
1276 |             }
1277 |         });
1278 |         avgAttendance = totalPct / totalCount;
1279 |     }
1280 |
1281 |     document.getElementById('rep-stat-total').textContent = totalCount;
1282 |     document.getElementById('rep-stat-avg').textContent = `${avgAttendance.toFixed(1)}%`;
1283 |     document.getElementById('rep-stat-low').textContent = lowAttendanceCount;
1284 | }
1285 |
1286 | // Render dynamic tables based on tab filters (All vs Deficient)
1287 | function renderReportsTable() {
1288 |     const headerRow = document.getElementById('table-reports-header');
1289 |     const tbody = document.getElementById('reports-tbody');
1290 |
1291 |     if (!headerRow || !tbody) return;
1292 |
1293 |     tbody.innerHTML = '';
1294 |     const students = StudentStore.getAll();
1295 |
1296 |     if (students.length === 0) {
1297 |         headerRow.innerHTML = '<th>Records</th>';
1298 |         tbody.innerHTML = `
1299 |             <tr>
1300 |                 <td style="text-align: center; color: var(--text-muted); font-style: italic; padding:
2rem;">
1301 |                     No student records registered in the system.
1302 |                 </td>
1303 |             </tr>
1304 |         `;
1305 |         return;
1306 |     }
1307 |
1308 |     if (activeReportsTab === 'all') {
1309 |         // Headers for All Students
1310 |         headerRow.innerHTML = `
1311 |             <th>Roll No</th>
1312 |             <th>Name</th>
1313 |             <th>Dept</th>
1314 |             <th>Total Classes</th>
1315 |             <th>Attended</th>
1316 |             <th>% Rate</th>
1317 |             <th>Eligibility</th>
1318 |         `;
1319 |
1320 |         students.forEach(s => {
1321 |             const pct = s.totalClasses > 0 ? (s.attendedClasses / s.totalClasses) * 100 : 0;
1322 |             const isDeficient = pct < 75.0;
1323 |
1324 |             const tr = document.createElement('tr');
1325 |             if (isDeficient) tr.className = 'row-low-attendance';
1326 |
1327 |             tr.innerHTML = `
1328 |                 <td><strong>#${s.rollNo}</strong></td>
1329 |                 <td>${s.name}</td>
1330 |                 <td><span style="background-color: rgba(255,255,255,0.05); padding: 0.2rem 0.5rem; border-
radius: 6px; font-size: 0.8rem;">${s.department}</span></td>
1331 |                 <td>${s.totalClasses}</td>

```

```

1332 |         <td>${s.attendedClasses}</td>
1333 |         <td><span class="badge ${isDeficient ? 'badge-danger' : 'badge-
success'}">${pct.toFixed(1)}%</span></td>
1334 |         <td><span style="font-weight: 600; color: ${isDeficient ? 'var(--color-danger)' : 'var(--
color-success)'}">${isDeficient ? 'Not Eligible' : 'Eligible'}</span></td>
1335 |     `;
1336 |     tbody.appendChild(tr);
1337 | });
1338 | } else {
1339 |     // Headers for Low Attendance Deficit View
1340 |     headerRow.innerHTML = `
1341 |         <th>Roll No</th>
1342 |         <th>Name</th>
1343 |         <th>Dept</th>
1344 |         <th>Attendance %</th>
1345 |         <th>Class Deficit</th>
1346 |     `;
1347 |
1348 |     let count = 0;
1349 |     students.forEach(s => {
1350 |         const pct = s.totalClasses > 0 ? (s.attendedClasses / s.totalClasses) * 100 : 0;
1351 |         if (pct < 75.0) {
1352 |             count++;
1353 |             // Deficit calculation: Math.ceil(0.75 * total - attended)
1354 |             const needed = 0.75 * s.totalClasses;
1355 |             const deficit = needed - s.attendedClasses;
1356 |             const classDeficit = deficit > 0 ? Math.ceil(deficit) : 0;
1357 |
1358 |             const tr = document.createElement('tr');
1359 |             tr.className = 'row-low-attendance';
1360 |             tr.innerHTML = `
1361 |                 <td><strong>#${s.rollNo}</strong></td>
1362 |                 <td>${s.name}</td>
1363 |                 <td><span style="background-color: rgba(255,255,255,0.05); padding: 0.2rem 0.5rem;
border-radius: 6px; font-size: 0.8rem;">${s.department}</span></td>
1364 |                 <td><span class="badge badge-danger">${pct.toFixed(1)}%</span></td>
1365 |                 <td><strong style="color: var(--color-danger);">${classDeficit} class${classDeficit > 1
? 'es' : ''}</strong></td>
1366 |             `;
1367 |             tbody.appendChild(tr);
1368 |         }
1369 |     });
1370 |
1371 |     if (count === 0) {
1372 |         tbody.innerHTML = `
1373 |             <tr>
1374 |                 <td colspan="5" style="text-align: center; color: var(--color-success); font-weight:
600; padding: 2rem;">
1375 |                     No deficient students. All students satisfy the 75% cutoff threshold!
1376 |                 </td>
1377 |             </tr>
1378 |         `;
1379 |     }
1380 | }
1381 | }
1382 |
1383 | // Render Chart.js Analytics Bar Chart
1384 | function renderAttendanceChart() {
1385 |     const ctx = document.getElementById('attendance-bar-chart');
1386 |     if (!ctx) return;
1387 |
1388 |     if (attendanceChartInstance) {
1389 |         attendanceChartInstance.destroy();
1390 |     }
1391 |
1392 |     const students = StudentStore.getAll();
1393 |     if (students.length === 0) {
1394 |         ctx.style.display = 'none';
1395 |         return;
1396 |     }
1397 |     ctx.style.display = 'block';
1398 |
1399 |     const labels = students.map(s => s.name);

```

```

1400 |     const data = students.map(s => s.totalClasses > 0 ? parseFloat(((s.attendedClasses / s.totalClasses) *
1401 | 100).toFixed(1)) : 0);
1402 |     // Dynamic coloring based on 75% boundary
1403 |     const backgroundColors = data.map(pct => pct < 75.0 ? 'rgba(244, 63, 94, 0.5)' : 'rgba(139, 92, 246,
1404 | 0.5)');
1405 |     const borderColors = data.map(pct => pct < 75.0 ? 'rgba(244, 63, 94, 1)' : 'rgba(139, 92, 246, 1)');
1406 |     attendanceChartInstance = new Chart(ctx, {
1407 |       type: 'bar',
1408 |       data: {
1409 |         labels: labels,
1410 |         datasets: [
1411 |           {
1412 |             label: 'Attendance Rate (%)',
1413 |             data: data,
1414 |             backgroundColor: backgroundColors,
1415 |             borderColor: borderColors,
1416 |             borderWidth: 1.5,
1417 |             borderRadius: 6,
1418 |             barPercentage: 0.6
1419 |           },
1420 |           {
1421 |             label: '75% Target Threshold',
1422 |             data: Array(students.length).fill(75),
1423 |             type: 'line',
1424 |             borderColor: 'rgba(251, 191, 36, 0.85)', // warning amber
1425 |             borderWidth: 2,
1426 |             borderDash: [5, 5],
1427 |             pointRadius: 0,
1428 |             fill: false
1429 |           }
1430 |         ]
1431 |       },
1432 |       options: {
1433 |         responsive: true,
1434 |         maintainAspectRatio: false,
1435 |         scales: {
1436 |           y: {
1437 |             min: 0,
1438 |             max: 100,
1439 |             grid: {
1440 |               color: 'rgba(255,255,255,0.04)'
1441 |             },
1442 |             ticks: {
1443 |               color: '#9ca3af',
1444 |               font: {
1445 |                 family: 'Plus Jakarta Sans',
1446 |                 size: 11
1447 |               }
1448 |             }
1449 |           },
1450 |           x: {
1451 |             grid: {
1452 |               display: false
1453 |             },
1454 |             ticks: {
1455 |               color: '#9ca3af',
1456 |               font: {
1457 |                 family: 'Plus Jakarta Sans',
1458 |                 size: 10
1459 |               }
1460 |             }
1461 |           }
1462 |         },
1463 |         plugins: {
1464 |           legend: {
1465 |             position: 'top',
1466 |             labels: {
1467 |               color: '#f3f4f6',
1468 |               font: {
1469 |                 family: 'Plus Jakarta Sans',
1470 |                 weight: '600',

```

```

1471 |         size: 11
1472 |     }
1473 | }
1474 | },
1475 | tooltip: {
1476 |     callbacks: {
1477 |         label: function(context) {
1478 |             return `${context.dataset.label}: ${context.raw}%`;
1479 |         }
1480 |     }
1481 | }
1482 | }
1483 | }
1484 | });
1485 | }
1486 |
1487 | // Entry Point Init
1488 | document.addEventListener('DOMContentLoaded', () => {
1489 |     initNavigation();
1490 |     updateDashboardStats();
1491 |     initStudentForm();
1492 |     renderDirectory();
1493 |     initSearchForm();
1494 |     initSortingSection();
1495 |     initAttendanceSection();
1496 |     initReportsSection();
1497 |     initSpotlight();
1498 |     initParticles();
1499 |     initCSVExport();
1500 | });
1501 |
1502 | // Spotlight Hover mouse tracking
1503 | function initSpotlight() {
1504 |     document.addEventListener('mousemove', e => {
1505 |         const cards = document.querySelectorAll('[data-glow]');
1506 |         cards.forEach(card => {
1507 |             const rect = card.getBoundingClientRect();
1508 |             const x = e.clientX - rect.left;
1509 |             const y = e.clientY - rect.top;
1510 |             card.style.setProperty('--mouse-x', `${x}px`);
1511 |             card.style.setProperty('--mouse-y', `${y}px`);
1512 |         });
1513 |     });
1514 | }
1515 |
1516 | // Particle Background Physics engine (CPU lightweight)
1517 | function initParticles() {
1518 |     const canvas = document.getElementById('particle-canvas');
1519 |     if (!canvas) return;
1520 |     const ctx = canvas.getContext('2d');
1521 |
1522 |     let width = canvas.width = window.innerWidth;
1523 |     let height = canvas.height = window.innerHeight;
1524 |
1525 |     window.addEventListener('resize', () => {
1526 |         width = canvas.width = window.innerWidth;
1527 |         height = canvas.height = window.innerHeight;
1528 |     });
1529 |
1530 |     const particleCount = Math.min(65, Math.floor(width / 20));
1531 |     const particles = [];
1532 |
1533 |     class Particle {
1534 |         constructor() {
1535 |             this.x = Math.random() * width;
1536 |             this.y = Math.random() * height;
1537 |             this.vx = (Math.random() - 0.5) * 0.3;
1538 |             this.vy = (Math.random() - 0.5) * 0.3;
1539 |             this.radius = Math.random() * 2 + 1;
1540 |         }
1541 |
1542 |         update() {
1543 |             this.x += this.vx;

```

```

1544 |         this.y += this.vy;
1545 |
1546 |         if (this.x < 0 || this.x > width) this.vx = -this.vx;
1547 |         if (this.y < 0 || this.y > height) this.vy = -this.vy;
1548 |     }
1549 |
1550 |     draw() {
1551 |         ctx.beginPath();
1552 |         ctx.arc(this.x, this.y, this.radius, 0, Math.PI * 2);
1553 |         ctx.fillStyle = 'rgba(139, 92, 246, 0.4)';
1554 |         ctx.fill();
1555 |     }
1556 | }
1557 |
1558 | for (let i = 0; i < particleCount; i++) {
1559 |     particles.push(new Particle());
1560 | }
1561 |
1562 | function animate() {
1563 |     ctx.clearRect(0, 0, width, height);
1564 |
1565 |     for (let i = 0; i < particles.length; i++) {
1566 |         const p1 = particles[i];
1567 |         p1.update();
1568 |         p1.draw();
1569 |
1570 |         for (let j = i + 1; j < particles.length; j++) {
1571 |             const p2 = particles[j];
1572 |             const dx = p1.x - p2.x;
1573 |             const dy = p1.y - p2.y;
1574 |             const dist = Math.hypot(dx, dy);
1575 |
1576 |             if (dist < 110) {
1577 |                 ctx.beginPath();
1578 |                 ctx.moveTo(p1.x, p1.y);
1579 |                 ctx.lineTo(p2.x, p2.y);
1580 |                 ctx.strokeStyle = `rgba(139, 92, 246, ${0.15 * (1 - dist / 110)})`;
1581 |                 ctx.lineWidth = 0.8;
1582 |                 ctx.stroke();
1583 |             }
1584 |         }
1585 |     }
1586 |     requestAnimationFrame(animate);
1587 | }
1588 | animate();
1589 | }
1590 |
1591 | // CSV Export bindings
1592 | function initCSVExport() {
1593 |     const exportBtnDir = document.getElementById('btn-export-csv');
1594 |     const exportBtnRep = document.getElementById('btn-export-csv-rep');
1595 |
1596 |     if (exportBtnDir) {
1597 |         exportBtnDir.addEventListener('click', exportToCSV);
1598 |     }
1599 |     if (exportBtnRep) {
1600 |         exportBtnRep.addEventListener('click', exportToCSV);
1601 |     }
1602 | }
1603 |
1604 | // Export logic to generate downloadable file
1605 | function exportToCSV() {
1606 |     const students = StudentStore.getAll();
1607 |     if (students.length === 0) {
1608 |         alert("No student records available to export.");
1609 |         return;
1610 |     }
1611 |
1612 |     let csvContent = "data:text/csv;charset=utf-8,";
1613 |     csvContent += "Roll Number,Name,Department,Total Classes,Attended Classes,Attendance Percentage\n";
1614 |
1615 |     students.forEach(s => {
1616 |         const pct = s.totalClasses > 0 ? ((s.attendedClasses / s.totalClasses) * 100).toFixed(2) : "0.00";

```

```
1617 |         const nameEscaped = s.name.includes(',') ? `"${s.name}"` : s.name;
1618 |         csvContent +=
`${s.rollNo},${nameEscaped},${s.department},${s.totalClasses},${s.attendedClasses},${pct}%\n`;
1619 |     });
1620 |
1621 |     const encodedUri = encodeURI(csvContent);
1622 |     const link = document.createElement("a");
1623 |     link.setAttribute("href", encodedUri);
1624 |     link.setAttribute("download", `student_attendance_report_${new Date().toISOString().slice(0,10)}.csv`);
1625 |     document.body.appendChild(link);
1626 |     link.click();
1627 |     document.body.removeChild(link);
1628 | }
```

students.txt

Data File — 6 lines

```
1 | # rollNo name department totalClasses attendedClasses
2 | 101 Alice_Smith CSE 40 35
3 | 102 Bob_Jones ECE 40 28
4 | 103 Charlie_Brown ME 40 38
5 | 104 Diana_Prince CSE 40 22
6 | 105 Ethan_Hunt IT 40 32
```